

Towards better data stores within the Dutch government

VNG Realisatie, project 'Uit betrouwbare bron'

Unfinished working draft, 13 July 2026

Unfinished working draft - Several chapters are outline only.

Everything in this document can – and probably will - change without notice.

Pre-publication as working draft for:

- Historie werkgroep of the NL API Strategie.
- Architects of the Common Ground Program
- Gemeente Utrecht

Preface

This document is a deliverable of the UBB project. See <https://www.uitbetrouwbarebron.nl>.

Status

This document is an unfinished working draft. The first six chapters are full concepts, the remaining chapters are stubs, holding only their planned subjects in outline.

Scope

This document is about the design of operational data stores: the core component of a register that records what the register 'knows'.

We place that store within its context: the register architecture. We sketch current stores and possible future stores. The focus of the document is on bitemporal stores that handle doubt (investigations) and provenance in a limited way. Our coverage of both is limited as well.

Analytical stores, for data-driven decision making - data warehousing, data lakes and the like - are out of scope.

Intended audience, assumed knowledge

We write for architects, designers and developers and assume basic knowledge of - Modelling (ERD, UML) - Relational databases - Temporal databases (https://en.wikipedia.org/wiki/Temporal_database); Jensen's survey gives a compact introduction (Jan.-Feb./1999), Snodgrass's book a thorough treatment (2000).

Related initiatives and information

Initiative	Link
NDS	https://www.digitaleoverheid.nl/nederlandse-digitaliseringsstrategie-nds/
IBDS	https://www.digitaleoverheid.nl/overzicht-van-alle-onderwerpen/data/interbestuurlijke-datastrategie/
FDS	https://federatief.datastelsel.nl/
GDI	https://www.digitaleoverheid.nl/overzicht-van-alle-onderwerpen/mido/generieke-digitale-infrastructuur-gdi/
UBB Project	https://www.uitbetrouwbarebron.nl

Tone and standing

The tone in this document is informal.

We do not claim to 'know'. It's a complex subject field where it is impossible to understand every situation.

We did, however, make a profound effort. We consulted seminal sources, considered alternatives, studied actual system designs and some of us have been deeply involved in the design and development of registers for years. So, it's not a lightly written document either.

The document is above all an invitation to explore this complex field together. Let's share what can work for all and be pragmatic where situations require different choices.

Table of contents

1	Introduction	5
2	Worked example	7
2.1	A person's history	8
3	The Store	10
3.1	S1 - The atemporal database	10
3.2	S2 - The unitemporal database - Variant A: System time	12
3.3	S3 - The unitemporal database - Variant B: Valid time	15
3.4	S4 - Bitemporal, interval-based	17
3.5	S5 - Bitemporal, point-based	19
4	Capture, process, account, publish	23
4.1	Capture	23
4.2	Process	23
4.3	Account	25
4.4	Publish	26
4.5	Transition	28
4.6	Notes	29
5	Provenance	30
5.1	The lineage, read backward	30
5.2	In practice	31
5.3	Corrections	32
5.4	Transition	32
5.5	Notes	33
6	Doubt	34
6.1	The flow	34
6.2	Data structure	35
6.3	The consumer's deviation	36
7	Bitemporal operations (STUB)	37
8	APIs (STUB)	38
8.1	Commands	38
8.2	Publication	39
9	Design part I - Concepts and building blocks (STUB)	41
10	Design part II - Table patterns (STUB)	42
11	Deferred topics (STUB)	43
12	Terminology	44

Appendix A: Bibliography	45
--------------------------------	----

1 Introduction

In this document we use the following working definition for a register:

A register is an application component - or a set of cooperating application components - for reliably recording and presenting an ordered collection of government information.

Two things in that definition shape the document. First, a register both 'records' and 'presents': it has a write side and a read side. Second, it is a set of cooperating components: the store that holds the data is one component, with others around it.

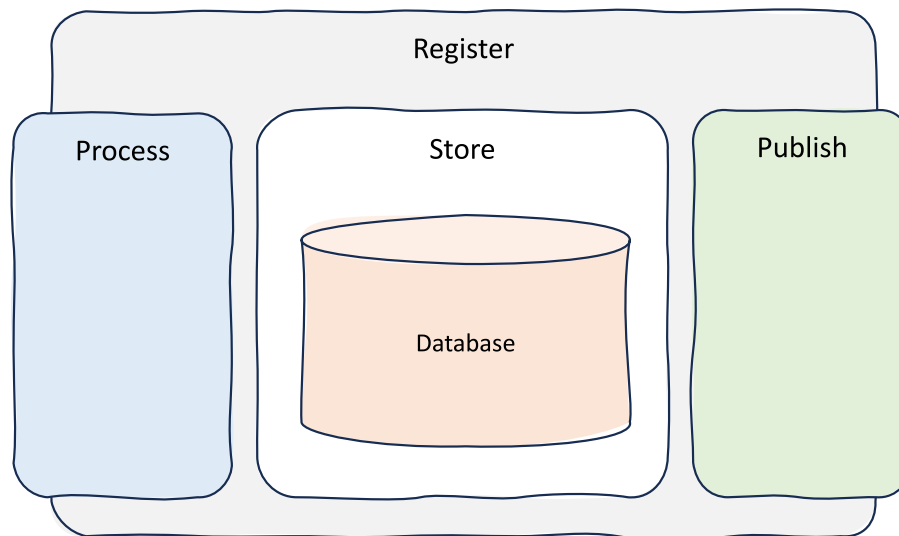


Figure 2.1: A register: the store at its centre, process on the write side, publish on the read side.

We first discuss the store on its own, then briefly the components around it, and then go back to the store to discuss details of the design.

The chapters:

2. The **worked example** introduces a register of persons that runs through the whole document: one person with twenty events.
3. **The store** is the heart of the document. Five store variants, from the ordinary database to the bitemporal store. The events are used to show what the different variants can express.
4. **Capture, process, account, publish** widens the view to the architecture around the store: how information is captured, processed into commands, accounted as effects, and published as projections.
5. **Provenance** covers the link between information in the register and its origin.
6. **Doubt** covers what happens when we doubt information: feedback reports, investigations, and how doubt attaches to information.
7. **Bitemporal operations** treats the operations systematically: what may happen to a row, and what each operation does.
8. **APIs** is about the register's interfaces: the command interface on the write side, notifications, information products and repeatable answers on the read side.
9. **Design part I - concepts** discusses the fundamental concepts behind a bitemporal store.
10. **Design part II - table patterns** discusses the different table patterns we need to build the store.

11. **Deferred topics** is a brief discussion of more complex topics whose full treatment is beyond the (current) scope of this document.
12. Finally the **terminology** chapter where every term is defined and weighed against possible alternatives.

2 Worked example

Throughout this document, we use one worked example: a register of persons. We chose it because it is easy to understand and rich. It is inspired by a civil-registration register.¹ It's drastically simplified, we have cherry-picked, made things up, and left things out. So don't use it as a reference in any way!

The main structure:

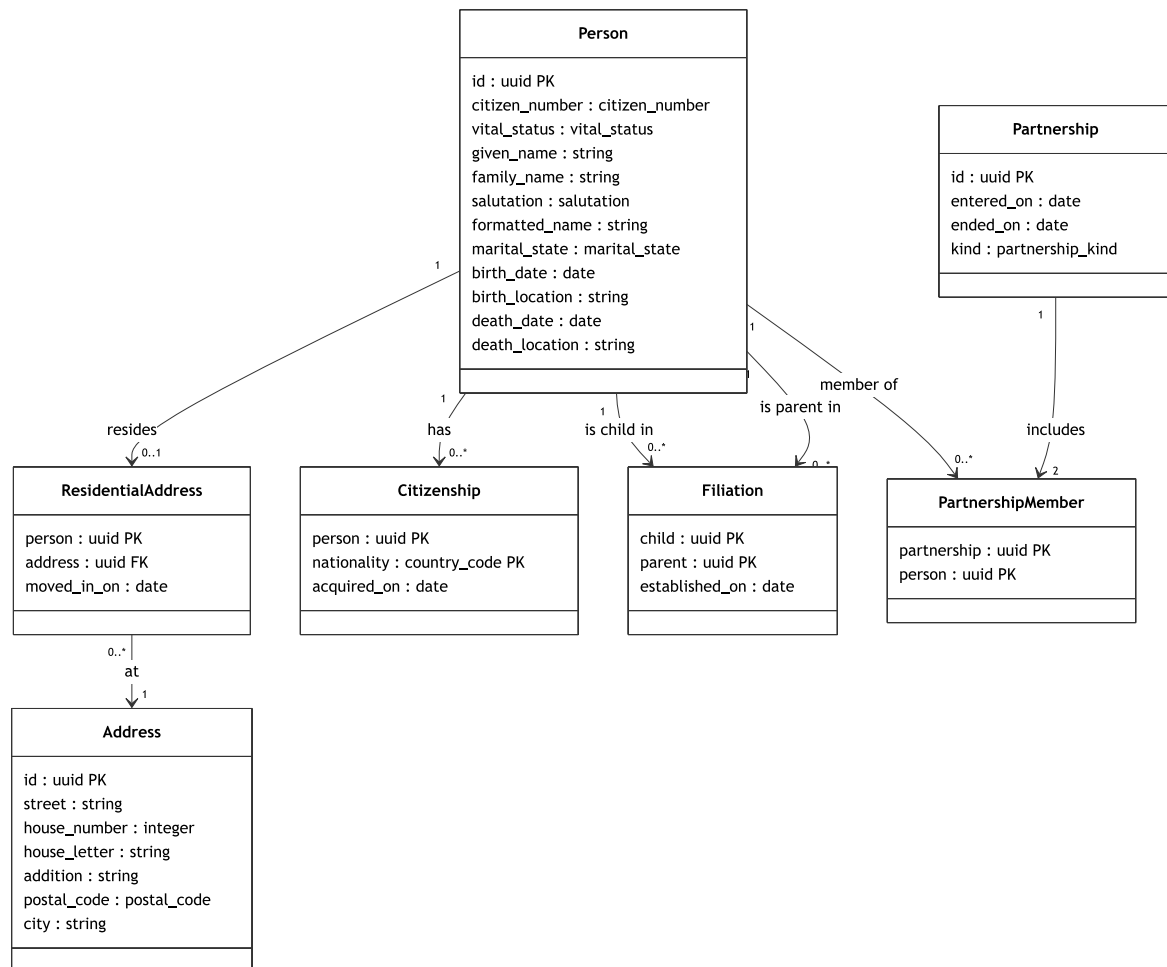
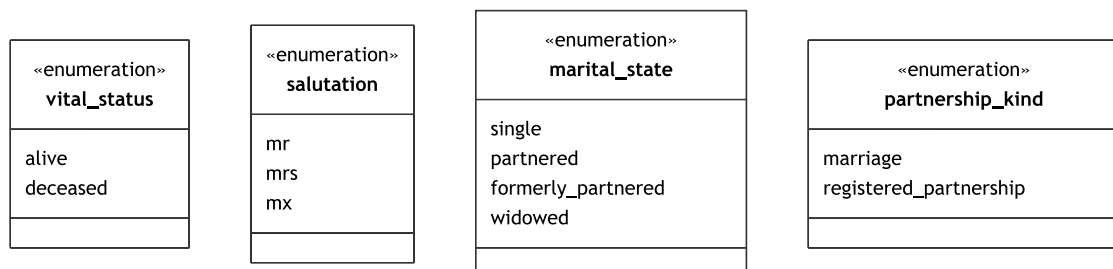


Figure 3.1: The data model of the worked example.

With the following enumerations:



¹ Inspired by the Logisch Ontwerp BRP (Rijksdienst voor Identiteitsgegevens, 2026).

Figure 3.2: The enumerations of the data model.

The named types are `citizen_number` (the person's national identifier), `postal_code` (a postal code, format left generic), `country_code` (a country, ISO 3166-1 alpha-2), and the usual `uuid`, `string`, `integer` and `date`.

2.1 A person's history

For the chapters ahead we need a rich example that goes with the structure. So we follow one person that we pull through a series of events that are unlikely to happen all in one person's life. The life of this person starts when she is left as a foundling at a hospital, and ends when he dies. The events are numbered e1 to e20, in the order the register learns of them.

We allow ourselves two liberties against the schema to make the example easier to read: - We don't use `uuids`, but simplified readable tokens like p5 and a2. - We don't use dates or times, but symbolic moments like T20.

If something happens in the domain at Tx, the register will always record it one chronon² later, so at Tx + 1. So T40 becomes T41.

Examples throughout this document, including character names, are drawn from Douglas Adams's *The Hitchhiker's Guide to the Galaxy* - they do not refer to real persons.

Event	Happened on	Concluded on	What happened
e1	T10		A newborn (p5) is found at the hospital. The parents are unknown and the birth date is an estimate. She's given the name Trillian Doe, registered to live at Marvin Mews, Guildford (a1), and she acquires British nationality (GB).
e2	T20		Trillian (p5) moves to Prefect Road, Rickmansworth (a2).
e3	T30		Trillian (p5) is adopted by Lunkwill and Fook McMillan (p2, p3), moves in with them at Beeblebrox Avenue, Islington (a3), and her family name is changed to McMillan.
e4	T40		Trillian (p5) moves to Fenchurch Street, London (a4).
e5	T50		Trillian (p5) emigrates. Where to is unknown: a foreign address is outside this register's scope.
e6	T40	T60	The emigration was a mistake. There was a mix-up of identities, Trillian (p5) never emigrated. The residence at a4 is reinstated.
e7	T76		Ahead of his wedding, Zaphod (p6) cannot produce a birth certificate: the archive of the town where he was born was destroyed by fire long ago. His birth data are recorded from a sworn statement.
e8	T80		Trillian (p5) marries Zaphod (p6).
e9	T92		A report from a foreign registry reaches the register: Zaphod (p6) may already have been married when he married Trillian (p5).
e10	T94		The register opens an investigation into the marriage.
e11	T100		The investigation finds that Zaphod (p6) was indeed already married, abroad, and never divorced. The court annuls his marriage to Trillian (p5) on that ground. Trillian (p5), who acted in good faith, retains the effects of the marriage; Zaphod (p6) retains none. The investigation is closed.
e12	T110		Trillian (p5) changes her gender to male; with it the given name becomes Arthur and the salutation mr.
e13	T112		Arthur (p5) requests that in normal situations the records read as if he was Arthur (mr) from birth.
e14	T120		Arthur (p5) enters a registered partnership with Ford (p7).

² Chronon after the temporal-database consensus glossary (Jensen et al., 1998).

Event	Happened on	Concluded on	What happened
e15	T130		Arthur (p5) moves to Milliways Lane, London (a6).
e16	T132		A report reaches the register: Arthur (p5) may never have lived at Fenchurch Street (a4).
e17	T134		The register opens an investigation into the past address of Arthur (p5).
e18	T35	T138	The investigation shows that Arthur (p5) never lived at Fenchurch Street (a4). The recorded move was wrong on two counts: the address should have been Country Lane, Cottington (a5), and it had already happened on T35, not T40. Note that Arthur (p5) did move to Milliways Lane (a6), on T130. So the change is only about his past location, not the present. The investigation is closed.
e19	T142		An archive-reconstruction project recovers the birth certificate of Zaphod (p6). It replaces the sworn statement as the source of his birth data.
e20	T150		Arthur (p5) dies at Stavro Mueller Plaza, London (a7). His record is frozen. The partnership ends: it is also Ford's (p7) record.

The Concluded on column is filled only for the two corrections, e6 and e18. An ordinary event needs no separate conclusion: it happens, the register hears of it, records it. A correction is different - it is itself a conclusion, reached later, about something further back: e6 concludes on T60 about the situation from T40 on; e18 concludes on T138 about the situation from T35 to T130.

How a store absorbs corrections - and what it remembers afterwards of what it once recorded - is where the variants of the store chapter differ.

For reference, a list of the addresses the events refer to:

id	street	house_number	house_letter	addition	postal_code	city
a1	Marvin Mews	42			H2G 1AA	Guildford
a2	Prefect Road	6			H2G 2BB	Rickmansworth
a3	Beeblebrox Avenue	29	a		H2G 3CC	Islington
a4	Fenchurch Street	3		II	H2G 4DD	London
a5	Country Lane	155			H2G 5EE	Cottington
a6	Milliways Lane	1			H2G 6FF	London
a7	Stavro Mueller Plaza	2			H2G 7GG	London

3 The Store

The store is the component that holds the data. We show it through the worked example - the register of persons - filled by the events of p5's history. In this chapter we visit five commonly seen variants, and watch what each one retains of those events.

3.1 S1 - The atemporal database

The 'ordinary' relational database.

The schema is normalised: fields are grouped into tables where each field has a clear functional dependence on the key.

The store holds the current state, nothing more. A change overwrites the previous value; ending a relationship deletes the row. In temporal literature this is called a 'snapshot relation': a table with no temporal support.³

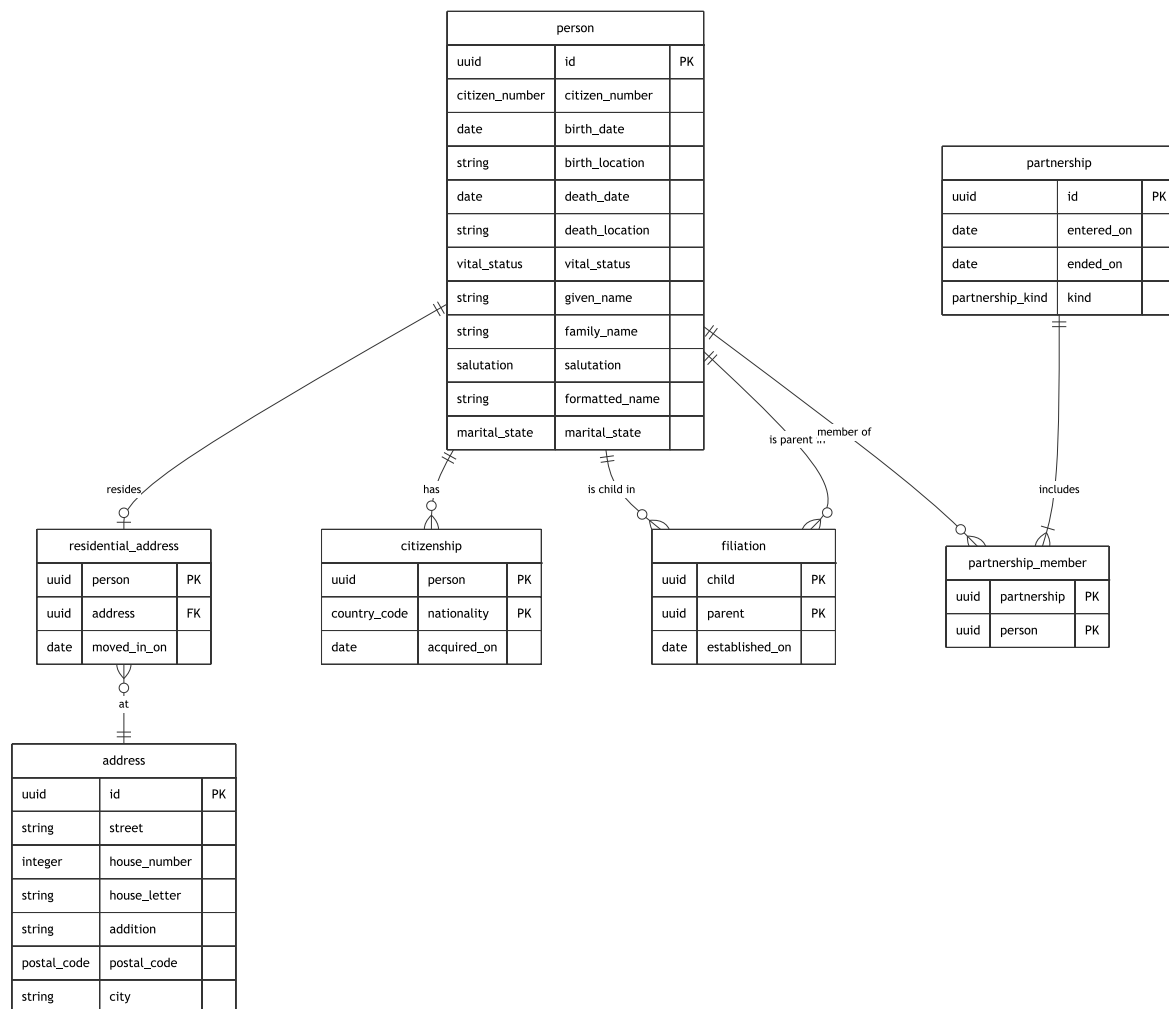


Figure 4.1: The S1 schema: the person as a single row - no time axis.

³ (Jensen et al., 1998)

3.1.1 Example

After all the events have happened, the store holds the following about p5:

person (one row, split in two to make it fit the page):

id	citizen_number	birth_date	birth_location	death_date	death_location	vital_status
p5	000000042	T10	Guildford	T150	London	deceased

id	given_name	family_name	salutation	formatted_name	marital_state
p5	Arthur	McMillan	mr	Mr. Arthur McMillan	partnered

residential_address:

person	address	moved_in_on
p5	a6	T130

citizenship:

person	nationality	acquired_on
p5	GB	T10

filiation:

child	parent	established_on
p5	p2	T30
p5	p3	T30

partnership and partnership_member:

id	entered_on	ended_on	kind
pt1	T80	T100	marriage
pt2	T120	T150	registered_partnership

partnership	person
pt1	p5
pt1	p6
pt2	p5
pt2	p7

Note that the register does not simply store the events, it processes them: a citizen number is assigned, the formatted name derived, etc.

3.1.2 The evaluation - what S1 can and cannot say

- The store only holds the last known values, no history.
- `given_name` now reads 'Arthur'; that it once read 'Trillian' has been overwritten and is gone.
- The store cannot express any form of doubt.
- The register once investigated p5's address, but the store cannot mark a value as doubted.
- The store cannot express any form of provenance.
- There are no links to events, processing, documents, etc.

3.2 S2 - The unitemporal database - Variant A: System time

In this variant, we add one time axis: *system time*.⁴

We limit the illustration to one relation: the residential address. The table is `his_residential_address`. The prefix marks it as a *historised* table.

his_residential_address		
uuid	record_id	PK
uuid	person	FK
timestamp	system_from	
timestamp	system_until	
date	moved_in_on	
uuid	address	FK

Figure 4.2: The S2 table: the address history under system time.

A change no longer overwrites the previous value; instead it closes the old row and appends a new one. The store keeps what it recorded, and when. This is the audit axis: it answers 'what did we hold, and since when'.

A store with a single time axis is called *unitemporal*.⁵ `system_from` opens a row, `system_until` closes it; an open `system_until` marks the register's *standing record*: the belief it currently holds.

The period is *half-open*: it includes the `from` moment and excludes the `until` moment - written $[T11, T21)$.⁶ So a row that closes at `T21` and its successor that opens at `T21` connect without gap and without overlap.

3.2.1 Example

⁴ The temporal literature calls this axis *transaction time* (Jensen et al., 1998; Snodgrass, 2000, p. 4); SQL:2011 and this document say *system time*. The choice is weighed in chapter 13.

⁵ ("Temporal Database," 2026)

⁶ (Snodgrass, 2000, pp. 89–91)

The contents of the table after all the events that impact the residential address have happened.

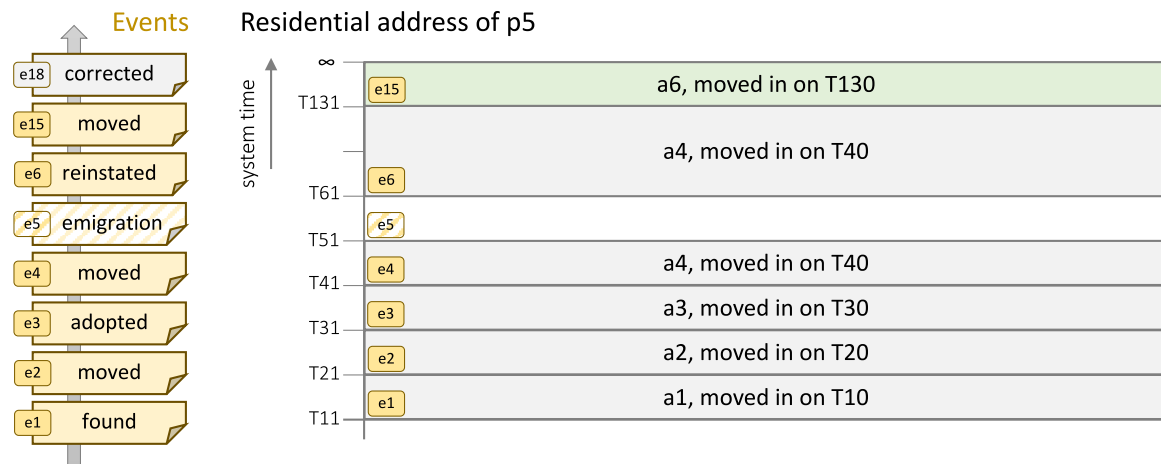


Figure 4.3: Each rectangle is a row, its height the row's system period: green the standing record, grey the rows that no longer stand; the white band is the gap.

Figure 4.3 shows the contents of the table in a diagram. Each rectangle represents a row in the table. To read the diagram in the order the rows entered the register, read it from bottom to top. The list at the left shows the events that impacted the address; the eX tag on a row is the event that wrote it. The events that could not be expressed are grey.

The corresponding rows in the `his_residential_address` table:

person	sys_from	sys_until	address	moved_in_on	comment
p5	T11	T21	a1	T10	e1 - Trillian (p5) is found and placed at Marvin Mews, Guildford (a1)
p5	T21	T31	a2	T20	e2 - Trillian (p5) moves to Prefect Road, Rickmansworth (a2)
p5	T31	T41	a3	T30	e3 - Trillian (p5) is adopted and moves in with the McMillans at Beeblebrox Avenue, Islington (a3)
p5	T41	T51	a4	T40	e4 - Trillian (p5) moves to Fenchurch Street, London (a4); the row is closed by the emigration, e5
p5	T61	T131	a4	T40	e6 - the emigration turns out to be a mix-up of identities and the residence is reinstated: Trillian (p5) still lives at Fenchurch Street (a4); closed by the move, e15
p5	T131		a6	T130	e15 - Arthur (p5) moves to Milliways Lane, London (a6); the standing record

We've added the *comment* column for clarification; it's not part of the actual table. `record_id` is omitted, and the system fields are abbreviated to `sys_from` / `sys_until`.

3.2.2 The evaluation - what S2 can and cannot say

- The store keeps every address it held, including the period in which it was held.
- System time is the register's timeline, not the domain's.
- p5 moved on T20; this is recorded on T21.
- between T51 and T61 the register held no address, though in hindsight p5 lived at a4 the whole time.
- A correction about a closed period cannot be recorded.

- Event e18 is a correction about the residency of p5 in the past. It doesn't change the most recent address, and cannot be recorded.
- 'Where did p5 live on T38?' has no answer.
- The store can say what it held on T38: a3. It cannot say where p5 lived on T38: that was a5, as the investigation later established.

The incorrect emigration (events e5 and e6) results in a gap in the timeline. That is correct: it shows the period in which the register incorrectly asserted that p5 had emigrated. That is useful information from an accountability perspective.

Residential address of p5

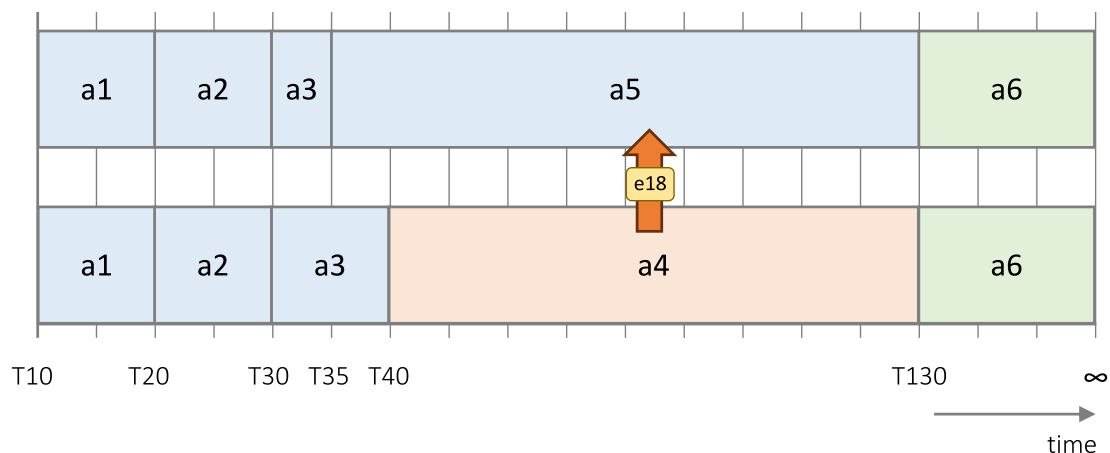


Figure 4.4: The correction e18: the address history as the register held it just before (below), and as corrected (above).

Figure 4.4 shows the correction. Both horizontal bars show the addresses where p5 has lived from left to right. The bar at the bottom shows the state right before the correction. The upper bar shows it after the correction. The result: p5 never lived at a4, but at a5, and that residence started on T35, not on T40.

The table cannot record this correction of the past. A new row - a new rectangle on top in Figure 4.3 - would change the current belief, not a prior one.

In some implementations this correction is forced into the history table by direct mutation. That defeats the whole purpose of the table as a reliable audit log.⁷

3.2.3 In practice

System time is standardised in SQL:2011 (system-versioned tables) and supported by several mainstream databases.⁸ The physical arrangement varies by product and practice, but it is one conceptual pattern in at least three common shapes:

- **One table holding everything** - the shape we show: `his_residential_address` carries the standing and the closed rows side by side.

⁷ (Snodgrass, 2000, p. 249)

⁸ (Böhlen et al., 2018, p. 2:1; Petković, 2016, p. 27)

- **A current table with a separate history table** - a plain `residential_address` holds only the standing records; a row that closes moves to the history table. Everyday queries read the small current table.
- **A current table with a generic audit log** - the fallback where native support is absent: triggers copy every closed row into one shared audit table (table name, timestamp, and the old row, often as a JSON blob). Cheap to add; asking what the register held at some earlier moment is hard.

All three shapes record the same facts. They differ in where the closed rows live, and in how much work the question ‘what did we hold on T38?’ takes.

3.3 S3 - The unitemporal database - Variant B: Valid time

In this variant, we start from S1 again and add the other time axis: *valid time* - the domain’s timeline.⁹

It records when the value is in effect in the domain (the world). The moments are determined and assigned by the domain, not by the system.

Like S2, this variant uses half-open periods: `valid_from` is included in the interval, `valid_until` is not. An open `valid_until` means that, to the best of the register’s knowledge, the value is still in effect.

The data type is often a date, not a timestamp: this corresponds to a granularity that matches the pace of the domain.

his_residential_address		
uuid	record_id	PK
uuid	person	FK
date	valid_from	
date	valid_until	
uuid	address	FK

Figure 4.5: The S3 table: the address history under valid time.

3.3.1 Example

The contents of the table after all the events happened:

⁹ (Jensen et al., 1998; Snodgrass, 2000, p. 4)

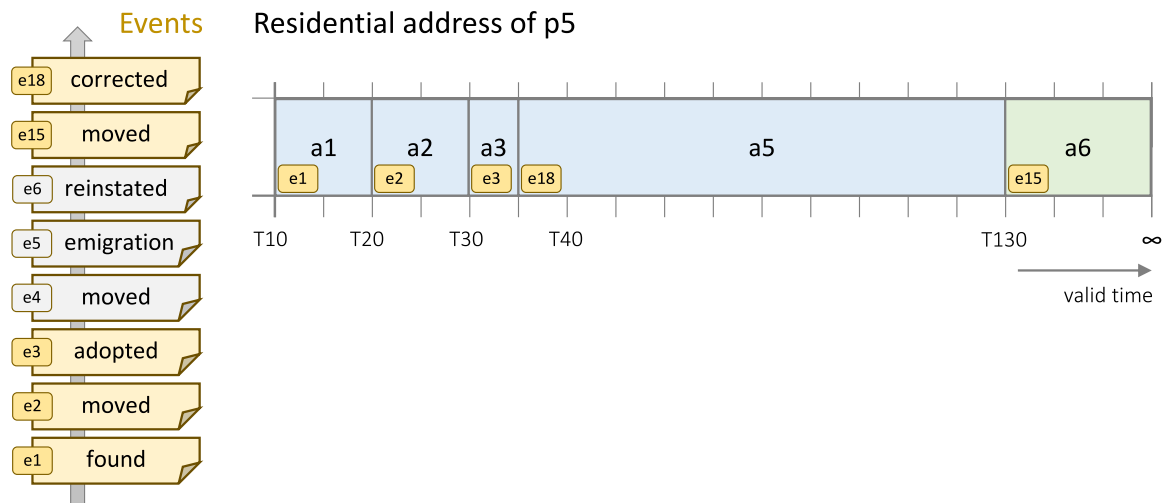


Figure 4.6: Each rectangle is a row, its width the row's valid period; green describes the present.

Five rows, displayed as rectangles. In the event list, e4, e5 and e6 are grey: what they once wrote is gone.

Residential address of p5

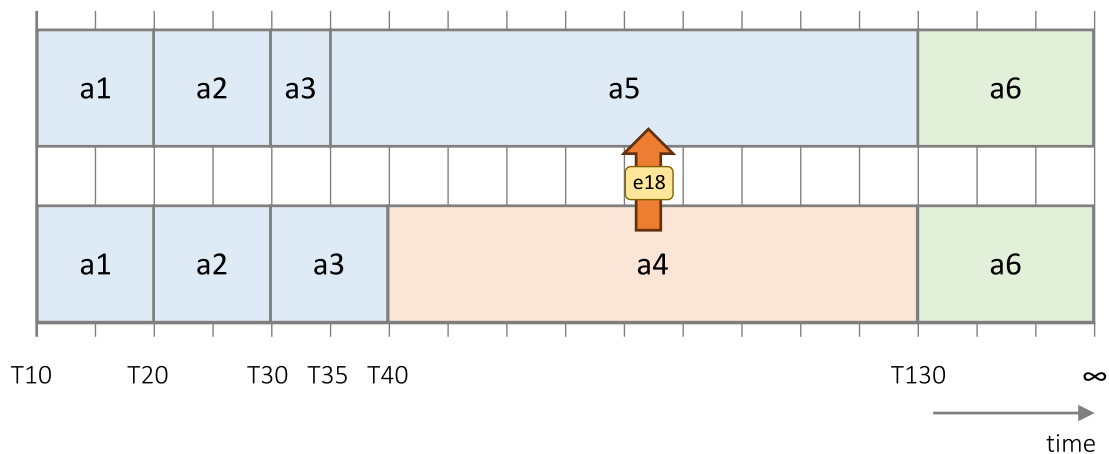


Figure 4.7: The correction e18 again: the table holds the corrected history (above) and nothing else; what the register held before (below) is gone.

Figure 4.7 shows the correction again. In S3 the table holds the upper bar. It only shows the state after the correction and keeps nothing of the state before.

The corresponding rows in the `his_residential_address` table:

person	val_from	val_until	address	comment
p5	T10	T20	a1	e1 - Trillian (p5) is found and placed at Marvin Mews, Guildford (a1); the move e2 ends the period on T20
p5	T20	T30	a2	e2 - Trillian (p5) moves to Prefect Road, Rickmansworth (a2); ended on T30 by the adoption, e3
p5	T30	T35	a3	e3 - Trillian (p5) is adopted and moves to Beeblebrox Avenue, Islington (a3); the end date was later corrected from T40 to T35, by e18
p5	T35	T130	a5	e18 - corrected: Arthur (p5) lived at Country Lane, Cottington (a5) from T35 until the move to a6

person	val_from	val_until	address	comment
p5	T130		a6	e15 - Arthur (p5) moves to Milliways Lane, London (a6) - still in effect

We've added the *comment* column for clarification; it's not part of the actual table. *record_id* is omitted, and the valid fields are abbreviated to *val_from* / *val_until*.

3.3.2 The evaluation - what S3 can and cannot say

- 'Where did p5 live on T38?' now has an answer.
- Corrections can be made, but only by overwriting: the audit log is gone.
- The answer to 'What did we hold on T45?' cannot be represented.

S2 and S3 fail in opposite ways. S2 answers 'what did we hold on T45?' but not 'where did p5 live on T38?'. S3 answers 'where did p5 live on T38?' but not 'what did we hold on T45?'.

3.3.3 In practice

Valid time is standardised in SQL:2011 as application-time period tables; product support is thinner than for system time.¹⁰

Valid time on its own is at home in fields that run on effective dates - HR, insurance, tax and tariff tables; Snodgrass's running case study is exactly this shape (2000, pp. 111–112). There the loss of accountability is typically solved by other means - a governed overwrite mode, paired with audit tables that keep the overwritten values.

3.4 S4 - Bitemporal, interval-based

In the bitemporal *interval-based* form, both axes are combined.¹¹

his_residential_address		
uuid	record_id	PK
uuid	person	FK
timestamp	system_from	
timestamp	system_until	
date	valid_from	
date	valid_until	
uuid	address	FK

¹⁰ (Petković, 2016, pp. 32–33)

¹¹ (Jensen et al., n.d., pp. 216–217)

Figure 4.8: The S4 table: both axes, interval-based.

3.4.1 Example

The contents of the table after all the events happened:

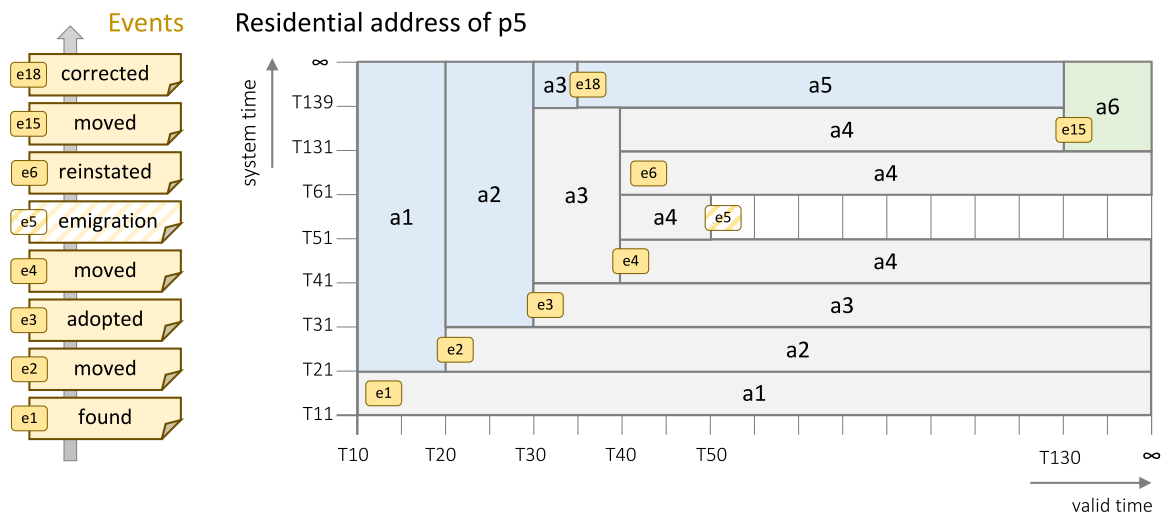


Figure 4.9: Each rectangle is a row: its height the system period, its width the valid period. Blue and green rows are standing, green also describes the present; white is absence.

Figure 4.9 draws the table as a plane: system time runs up, valid time runs right. Bottom to top is the order the register recorded the rows. The top edge holds the standing rows, read from left to right. Every event could be expressed, so none is grey.

We see a normal update pattern in the lower-left corner, repeated three times. We see each of these addresses two times. First as a row that stretches to infinity. It expresses 'p5 lives there, no end known yet'. Then the change that ended the residence, followed by a new assertion, again until infinity.

This pattern breaks when the emigration kicks in. The table cannot express foreign addresses, so a gap appears. The correction reinstates the residence, but the gap remains. There was a period when the register falsely asserted that p5 had emigrated; nothing can erase that.

Finally, on the top row, the retroactive correction: the realisation that p5 never lived at a4, but at a5, from T35.

When we list the corresponding rows of the `his_residential_address` table, the value of bitemporal diagrams becomes clear. The diagrams are hard to read, but the table is - for humans - nearly impossible to read.

person	sys_from	sys_until	val_from	val_until	address	comment
p5	T11	T21	T10		a1	e1 - a1 from T10, no end known yet; superseded when e2 supplies one
p5	T21		T10	T20	a1	e2 - a1 now known to end at T20; standing
p5	T21	T31	T20		a2	e2 - a2 from T20, open; superseded by the move e3
p5	T31		T20	T30	a2	e3 - a2 ends at T30; standing
p5	T31	T41	T30		a3	e3 - a3 from T30, open; superseded by the move e4
p5	T41	T139	T30	T40	a3	e4 - a3 ends at T40; superseded by the correction e18
p5	T41	T51	T40		a4	e4 - a4 from T40, open; superseded by the emigration e5

person	sys_from	sys_until	val_from	val_until	address	comment
p5	T51	T61	T40	T50	a4	e5 - a4 ends at T50 (emigration); superseded when e6 reinstates a4
p5	T61	T131	T40		a4	e6 - a4 reinstated, open again; superseded by the move e15
p5	T131	T139	T40	T130	a4	e15 - a4 ends at T130; superseded by the correction e18
p5	T131		T130		a6	e15 - a6 from T130; standing
p5	T139		T30	T35	a3	e18 - a3's end corrected to T35; standing
p5	T139		T35	T130	a5	e18 - a5 from T35 to T130, the corrected address; standing

Again the *comment* column is only for clarification; it's not part of the actual table. *record_id* is omitted, and the time-axis fields are abbreviated.

3.4.2 The evaluation - what S4 can and cannot say

- Both temporal questions now have an answer.
- 'Where does p5 live?' - a6.
- 'What did the register hold on T45 about where p5 lived?' - a4.
- Full 'time travel' is possible.
- 'Where did p5 live on T50, according to what the store held on T135?' - a4.
- Corrections no longer destroy anything.
- The cost is the increased complexity and number of rows.

3.4.3 In practice

Both axes are standardised in SQL:2011: a system-versioned table combined with an application-time period table is the standard's bitemporal form. Products that carry both axes are rarer than products that carry system time alone.

3.5 S5 - Bitemporal, point-based

In the bitemporal *point-based* form, both axes are still present, but the periods are replaced with points.

his_residential_address		
uuid	record_id	PK
uuid	person	FK
timestamp	system_from	
date	valid_from	
uuid	address	FK

Figure 4.10: The S5 table: both axes, point-based.

3.5.1 Example

The contents of the table after all the events happened:

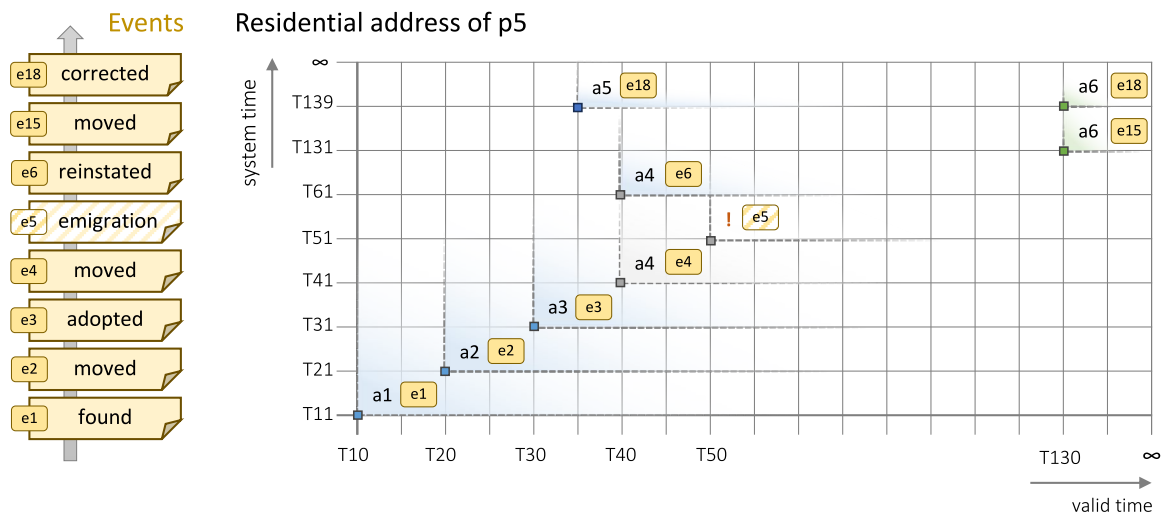


Figure 4.11: Each row in the history table is a point in the diagram.

Figure 4.11 draws the table still in the plane. Again the system time runs up, the valid time runs right. The difference is that each record only holds the `system_from` and the `valid_from`. So instead of rectangles we have points. The diagram still suggests rectangles - with a fade - because that is how the points are interpreted.

The corresponding rows in the `his_residential_address` table:

person	sys_from	val_from	address	comment
p5	T11	T10	a1	e1 - a1 from T10
p5	T21	T20	a2	e2 - a2 from T20
p5	T31	T30	a3	e3 - a3 from T30
p5	T41	T40	a4	e4 - a4 from T40
p5	T51	T50		e5 - emigration: no address from T50
p5	T61	T40	a4	e6 - a4 reinstated from T40, its true start
p5	T131	T130	a6	e15 - a6 from T130
p5	T139	T35	a5	e18 - correction: a5 from T35
p5	T139	T130	a6	e18 - a6 reasserted from T130, its unchanged start

The correction e18 writes two rows. A new point rewrites the history from its start onward, and whatever lies beyond that start must be written again, or it is gone. So the second row reasserts a6. Without it, the standing history would end on an open-ended a5, and the move to a6 would have vanished.

Again the `comment` column is only for clarification; it's not part of the actual table. `record_id` is omitted, and the time-axis fields are abbreviated.

3.5.2 The evaluation - what S5 can and cannot say

- Which rows still stand is derived, not stored.
- A row no longer reads on its own - its end, and whether it ended or was corrected, are derived from other rows; that it still stands, from the absence of them.
- Absence is a row of its own.
- The emigration is the empty row at T50: `valid_from` set, no address.

In the point-based form nothing is ever closed in place; an ending must arrive as a row of its own. We call it a *marker row*: a row that asserts no new value but marks what happened to an old one. The emigration's empty row above is one - no address from T50. Where the value is part of the key - a nationality, a parent - the marker row keeps the key and says 'ended' in a field. A marker row can carry more than the ending: also that a value was wrong, or is hidden - the reason it left the current view.

3.5.3 In practice

The Dutch base registers cover nearly the whole design space. The BAG and the BRK carry intervals on both axes¹²; the Handelsregister and the WOZ close only the valid-time interval and keep a single registration timestamp¹³; the BGT and the BRT version only along the registration axis - the BGT states outright that it does not record valid-time history¹⁴; the BRP is point-based on both axes, deriving every end from the ordering of single dates¹⁵; and the BRO's architecture requires both histories, with the field-level shape still to come.¹⁶

The temporal-database literature consistently favours the interval-based variant and names the costs of the point-based variant:

- Date treats the point-based design as deficient: a table with a single 'since' timestamp cannot record when a value stopped and is "still only semitemporal" (2014, pp. 77–78). His remedy is the interval; the marker row is the workaround for those who stay.
- Böhlen and colleagues name a second cost: two adjacent equal periods cannot be told from one long period. You can store the two rentals as two rows, but under point-based semantics the interval is not part of the fact - only the days it covers are - so "the two consecutive 2-day rentals disappear: they are merged into a single 4-day rental" unless an extra attribute, a contract number, holds them apart; interval-based models "are more expressive" and keep them apart without help (2017, pp. 56–57).
- Snodgrass names an operational cost: requiring a contiguous history "is a nonsequenced constraint" (2000, p. 130) - a constraint checked across the whole table, not row by row.

The point-based form has a natural home: analytics. The contiguous, gap-free timeline is the norm in dimensional modelling (slowly changing dimensions (Kimball & Ross, 2013); Anchor Modeling (Rönnbäck et al., 2010)).

¹² (Kadaster, 2018, Blad 5-6; Kadaster, 2025, pp. 5, 14)

¹³ (Kamer van Koophandel, 2022; Waarderingskamer, 2024, pp. 46–48)

¹⁴ (Geonovum, 2020, pp. 26–27; Kadaster, 2026, p. 29)

¹⁵ (Rijksdienst voor Identiteitsgegevens, 2026, pp. 178–179)

¹⁶ The BRO evidence is its programma start architectuur (Basisregistratie Ondergrond, 2017, pp. 53–54), not a gegevenscatalogus: materiële and formele historie are required as principles; the field design is deferred to the IMBRO catalogue.

It ensures that a query always finds a row: every instant maps to exactly one version, and a join lands on exactly one row. The cost is that there must always be a value - absence itself becomes a value, for example an 'unknown' or 'none' added to the enumeration.

So in analytics the gap-free timeline is a feature. For a register, in line with the literature, we do not recommend the point-based form for new implementations.

4 Capture, process, account, publish

The introduction showed a very simple picture of the register, with the focus on the store. This chapter discusses the environment of that store: the components around it, and what travels between them. The treatment stays brief - the surroundings are context, the store remains the subject.

The ideas in this chapter are the subject of the Guide produced by the project Uit betrouwbare bron (VNG, 2026).

4.1 Capture

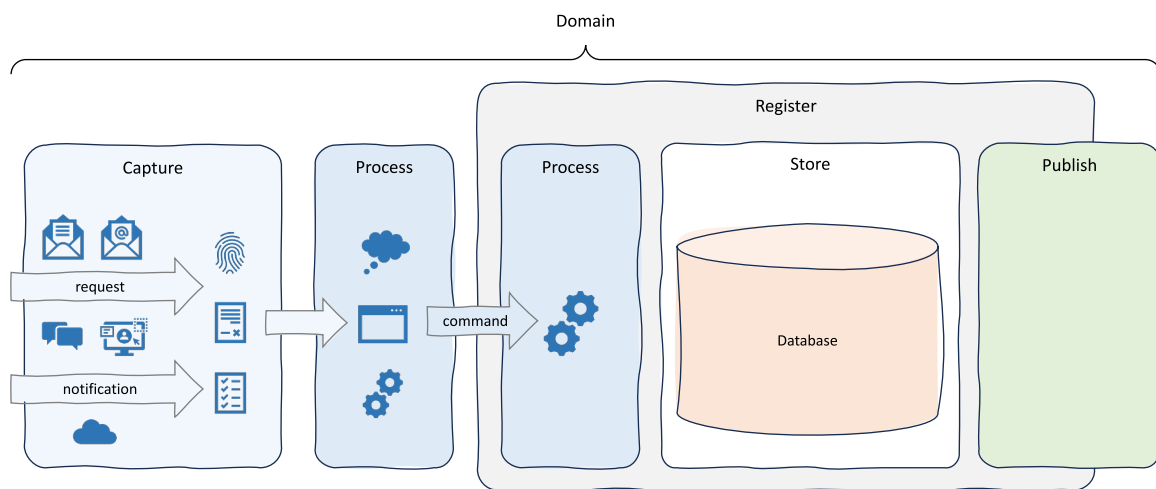


Figure 5.1: The register in its surroundings.

It all starts at the left side of the image. Requests and notifications arrive in all shapes: letters, emails, conversations, chats, interactions on websites, messages from other systems, etc. Capture translates them into a form processing can handle. The input is harmonised and the parties behind it are identified; documents may be drawn up, form engines may play a part.

All of it feeds into processing: people working with applications, the applications automating what they can. From there the information reaches the store, as commands.

Where the register ends is a debate this document stays out of. In the figure the boundary is drawn around the store and its immediate components; the brace on top spans everything that belongs to the domain. Some would call all of that the register. For this chapter the parts matter, not the border.

4.2 Process

Process appears twice in the picture. The part outside the register is the register's client; the part inside is the server side. Between them travel commands.

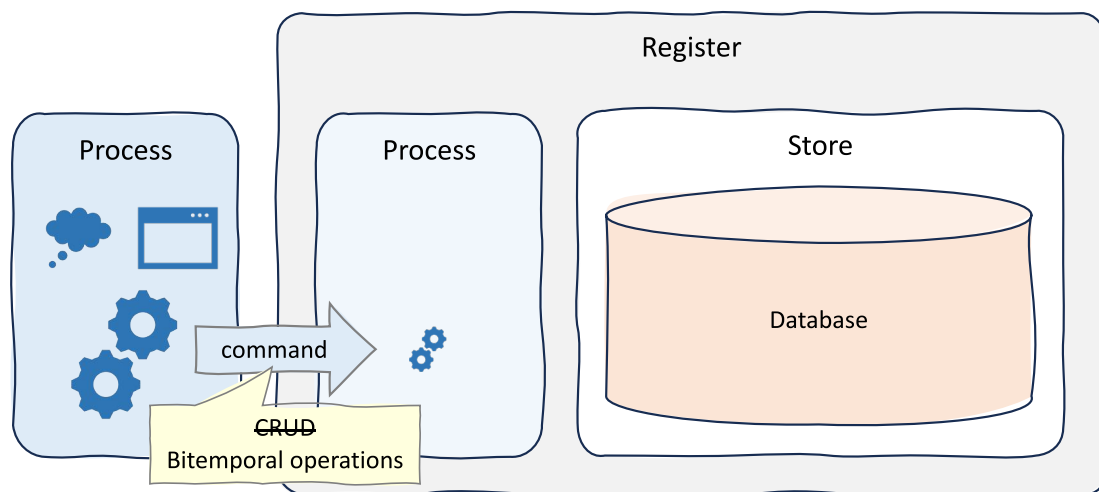


Figure 5.2: A CRUD command set: the intelligence sits with the client, the server does little more than table operations.

Modern APIs often speak CRUD: create, read, update, delete. With a bitemporal store inside, the commands must at least match the store's bitemporal structure, so we need bitemporal operations. This is the subject of chapter 8.

Even then, the intelligence sits in the client. That can work while the client is one piece of software, under the absolute control of the register. If it's not, this way of working becomes a serious risk. Several of the Dutch core registrations have different client systems, made by different vendors. On top of that, the different organisations that operate them use different conventions that do seep through into the data inside the historical tables. The result: The data become hard to interpret, the patterns both vendor- and sometimes organisation-specific.¹⁷

The conversion of the old civil register (the GBA) into its successor (the BRP) made the cost visible. Not all temporal data could be interpreted automatically; human experts had to establish what the rows meant (Ministerie van Binnenlandse Zaken en Koninkrijksrelaties, 2017, p. 124).

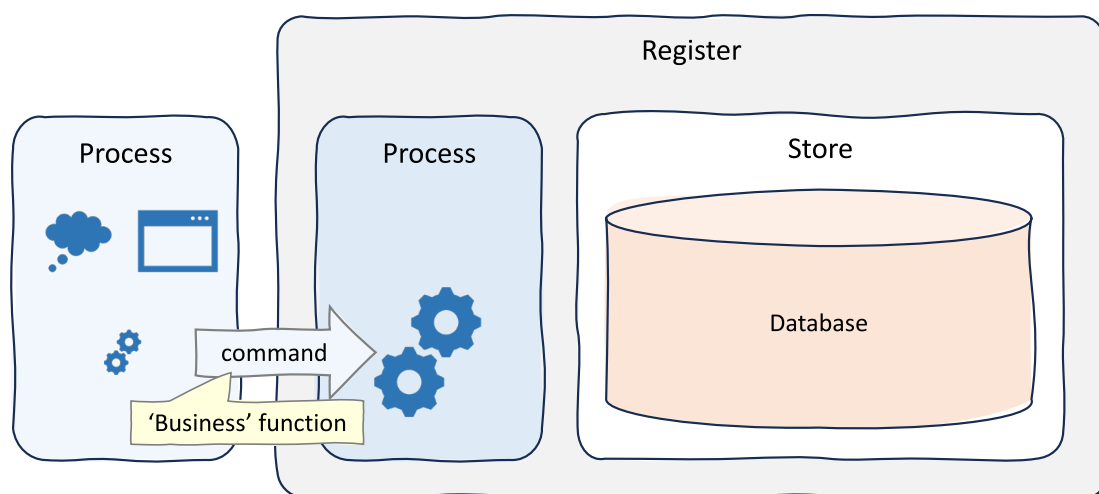


Figure 5.3: Business functions: the processing has moved into the register, and the command carries the intent.

¹⁷ (A. Widlak, 2022, pp. 87–88, 183)

The way out is to move a significant part of the processing into the register, to the server side. With that move the commands change in nature: they become business functions - register a birth, correct an address. In the figure the client's gears have shrunk; the server's have grown. A command now carries meaning - intent - and a lot of business value sits in it.

4.3 Account

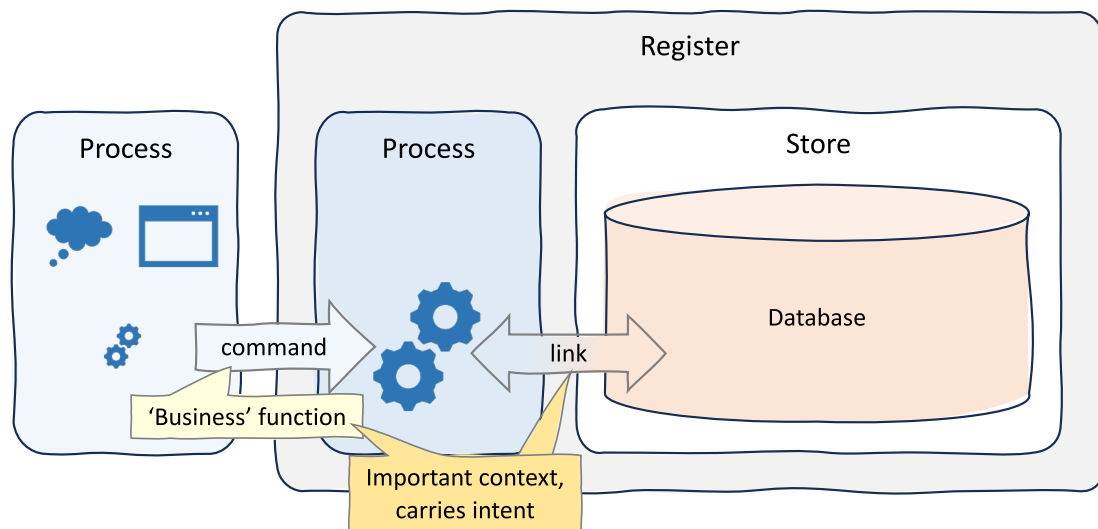


Figure 5.4: The command kept: the information in the database stays linked to the command that brought it there.

Build the register this way, and the next wish follows on its own: link the information in the database to the command that brought it there. The command tells why the information is there; it becomes a provenance artefact.

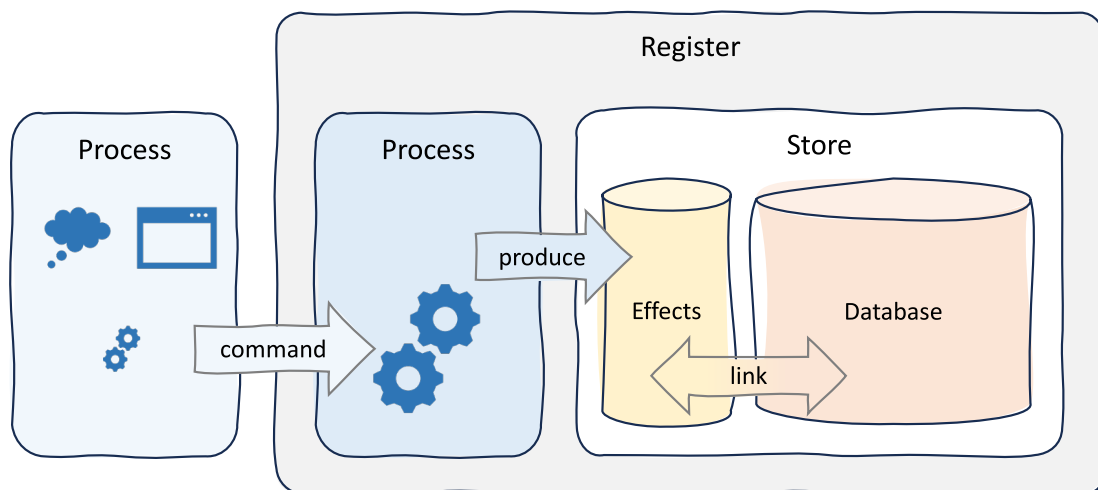


Figure 5.5: Not the command but its effect: the process produces effects, and the database links to those.

Yet the command is a bridge between two worlds. On one side the outside world, where the information was captured and the intent lives. On the other the inside of the register: algorithms that take the information, match semantics, reach decisions. Both sides matter for understanding why information is in the database, and the command covers only one - it does not tell how it was processed. So the processing settles its outcome in a durable artefact of its own: the effect - and it is the effect the database links to.

Often the difference looks trivial: the command is imperative, the effect past tense - 'correct this address' becomes 'this address was corrected'. Even then the effect records a decision, however small: the information from the command was accepted into the register.

Often there is more. Semantics might shift on the way in - what a term means outside the register is mapped onto what it means in the domain. That mapping is a context transition, and it deserves deliberateness on both sides of the register.¹⁸

Apart from the context transition, values are derived, calculations are performed. All of this comes together in the recorded effect.

The effects get a store of their own, append-only: effects are added, never changed, never removed. That store is the register's account of what it did. For the effect itself, the UBB project gives a working definition (VNG, 2026):

A significant statement, made by an actor authorised to make it in carrying out a task, which within a given context forms a basis for follow-up actions - and on which those involved can hold the producing actor to account.

4.4 Publish

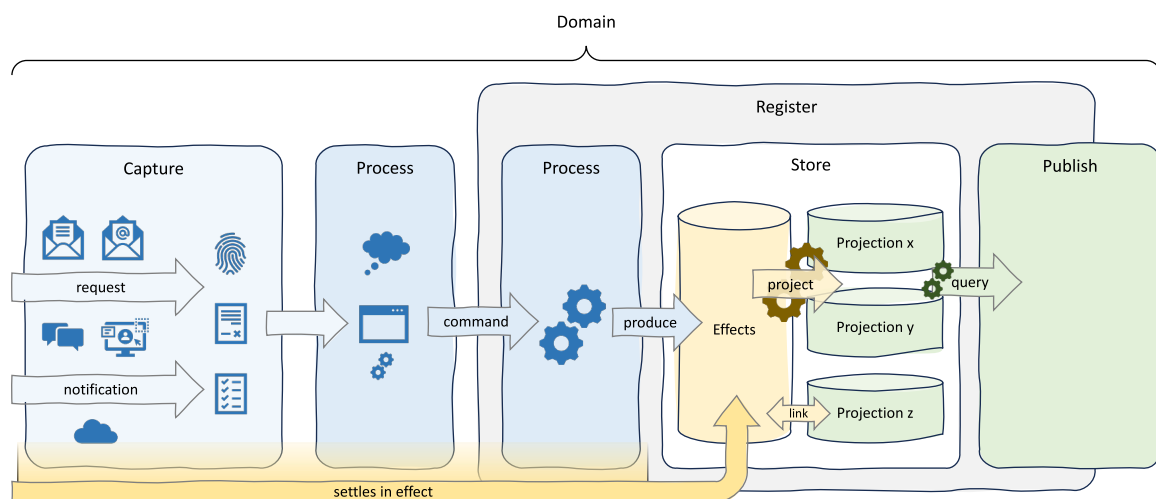


Figure 5.6: The full picture: the monolithic database has given way to projections; effects are projected into them, and the publish side queries them. Along the bottom: everything from capture through process settles in the effect.

In the final picture the database has given way to a set of projections, each derived from the effects. The band along the bottom sums the previous section: everything from capture through process settles in the effect.

Event sourcing knows the projection pattern - often with one difference. Event sourcing treats a projection as disposable: throw it away, replay the effects, and it returns.¹⁹ Our projections often have to be persistent.

The reason is the 'repeatable question'. The previous chapter called it 'time travel' and asked: 'What did the register hold on T45 about where p5 lived?' Whoever asks that question - now or years from now - must get the answer the register gave then.

¹⁸ That context transitions matter is shown in Peeters & Widlak (2018, pp. 180–181): an imperfect mapping - organisations took the civil registry's residency concept for their own eligibility rule - cost people their health insurance and, in practice, their right to vote.

¹⁹ (Overeem et al., 2021, p. 9)

A projection that is derived and nothing more cannot promise that. Algorithms change; bugs get fixed. Replay the effects through the changed code, and some answers change with it.²⁰ Then the register ‘lies’ about what it once answered, and that is often not acceptable.

So a projection stores its answers rather than re-deriving them: every state it ever held is kept. While the purpose of such stores is different - focused only on the projection - the structure is not. A bitemporal store is well suited to capture that state. The separation of concerns - separating processing and accounting from publishing - makes their design somewhat easier as we can optimise both for their purpose instead of getting a less optimal compromise of both.

The image below shows why this separation matters:

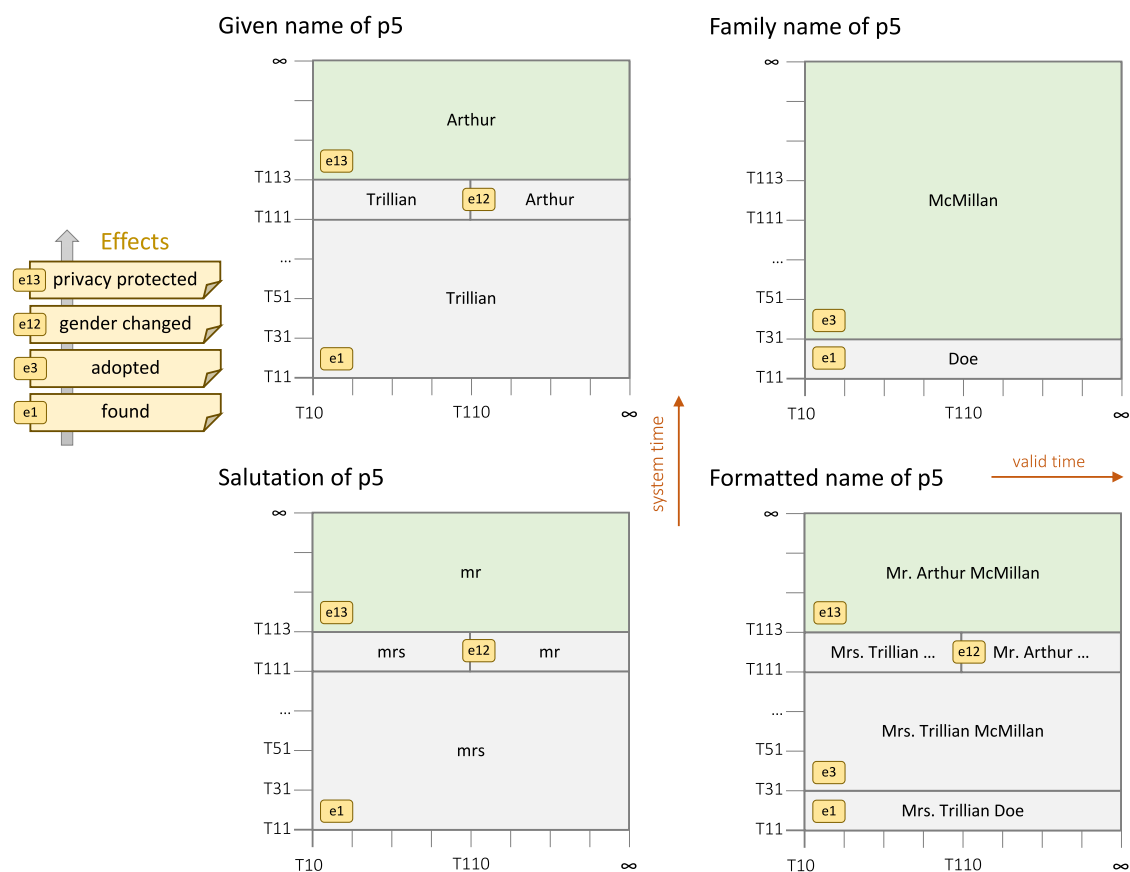


Figure 5.7: Four fields of p5, each as its own bitemporal plane. The formatted name is a projection over the other three.

Figure 5.7 shows the effects on the left and four fields next to them: the given name, the family name, the salutation, and the derived field formatted name, which typically lives in a projection. The non-derived fields are information atoms: they only hold one piece of information. This decomposition is known as sixth normal form (6NF).²¹ Each effect touches its own fields: the first registration (e1) sets every value, the adoption (e3) changes only the family name, the gender change (e12) the given name and the salutation.

Had we combined those into a single bitemporal table - as is common practice today - the rows would duplicate all the values. One could argue this redundancy could be controlled with a strict bitemporal implementation on top of the tables. But even then, knowing what changed requires comparing rows and

²⁰ (Müller, n.d., pp. 22–23; Overeem et al., n.d., p. 201)

²¹ (Date et al., 2003)

finding differences. This becomes even more pronounced when we add provenance fields (chapter 6). Such fields 'govern' the whole row, but only justify the fields that changed. Whether this is acceptable or not is a design choice. We will return to this topic in the first design chapter (chapter 10).

The projections regroup the information: the formatted name combines given name, family name and salutation - 'Mrs. Trillian Doe' at the start, 'Mr. Arthur McMillan' by the end - ready for addressing the person. Its plane has the most rows: it changes whenever any of its parts changes. Obviously this row cannot carry detailed provenance - that is by design. If a consumer needs to know when the given name changed, it needs a different projection - one that does contain the information 'atoms'.

By separating the effects from the projections, the architecture now follows the CQRS pattern (Command Query Responsibility Segregation)²² not only in the APIs but down into the design of the data stores.

Reduced to its movements, the architecture reads as four verbs.

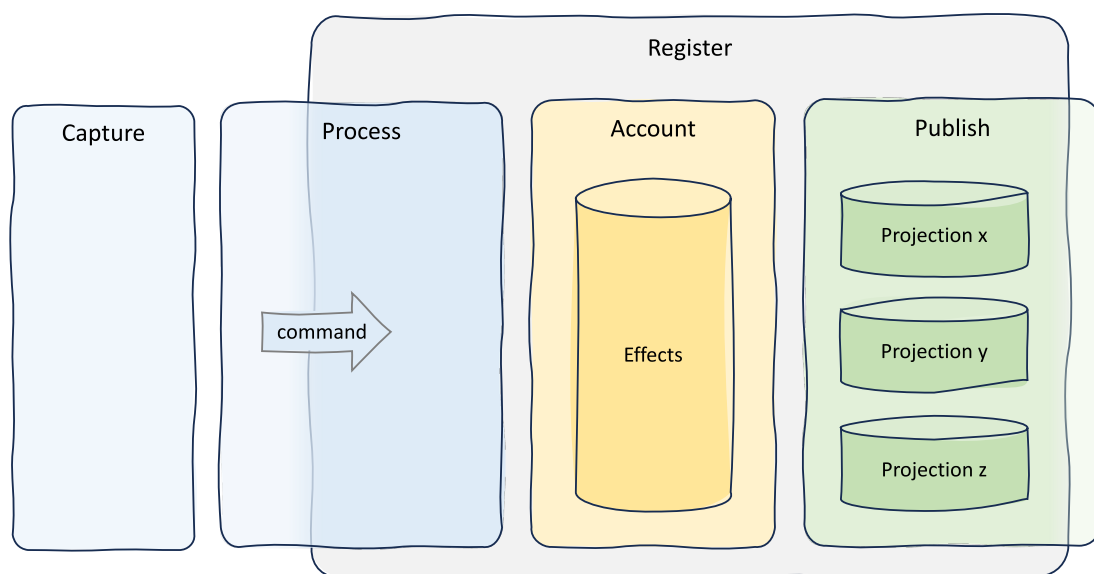


Figure 5.8: The four movements: capture, process, account, publish.

Figure 5.8 closes the walk. Capture stays outside the register. The boundary cuts through process: how much falls inside is the design choice this chapter walked, from bare bitemporal operations to business functions. And what the earlier pictures called the store falls apart into two movements: account holds the effects, publish holds the projections.

4.5 Transition

The full architecture does not have to arrive at once. There is a route, and it retraces this chapter's figures.

1. Start with the bitemporal store of the previous chapter. Trade the bare bitemporal operations for business functions.
2. Link the rows to the commands, as provenance.
3. Then store the effects and link them to the store's rows - still without projections.
4. The last step is the full picture: multiple projections, derived from the effects.

²² (Fowler, 2011; Young, 2010)

The Guide from the project 'Uit betrouwbare bron' tells more about registers built along these lines - www.uitbetrouwbarebron.nl.

4.6 Notes

Three notes for the careful reader.

Append-only, twice. The journal and the store are both append-only, under different disciplines. The journal never changes at all: a correction is a new effect, and no field of a recorded effect is ever touched. The store appends and closes: its one controlled write is setting `system_until`. Both refuse to destroy.

Availability is not a third axis. 'When could a consumer see it' sounds like a third timeline next to the two of the previous chapter. It is not.²³ The projection is a store of its own, with its own system time - and that clock, read by a consumer, tells when the data was available.

Another system's clock is data. System time is owned by its system. The journal stamps an effect with its own clock; when the effect reaches a projection - or another register - that stamp travels along as data. One system's axis is the next system's data. That is why no third axis is needed.

²³ The literature weighed a third axis of its own - decision time - and found the case for supporting it unclear (Jensen & Snodgrass, Jan.-Feb./1999, p. 37).

5 Provenance

Provenance answers questions like: why is this information in the register? How did it get here? What is the source?

Provenance is a substantial field of its own; the Dutch *Handreiking Datalineage* covers it in more detail (NORA Expertgroep Gegevensmanagement, 2025), and the W3C's PROV standards give it a formal model (W3C, 2013). What follows below is an exploration that shows how it impacts our store.

5.1 The lineage, read backward

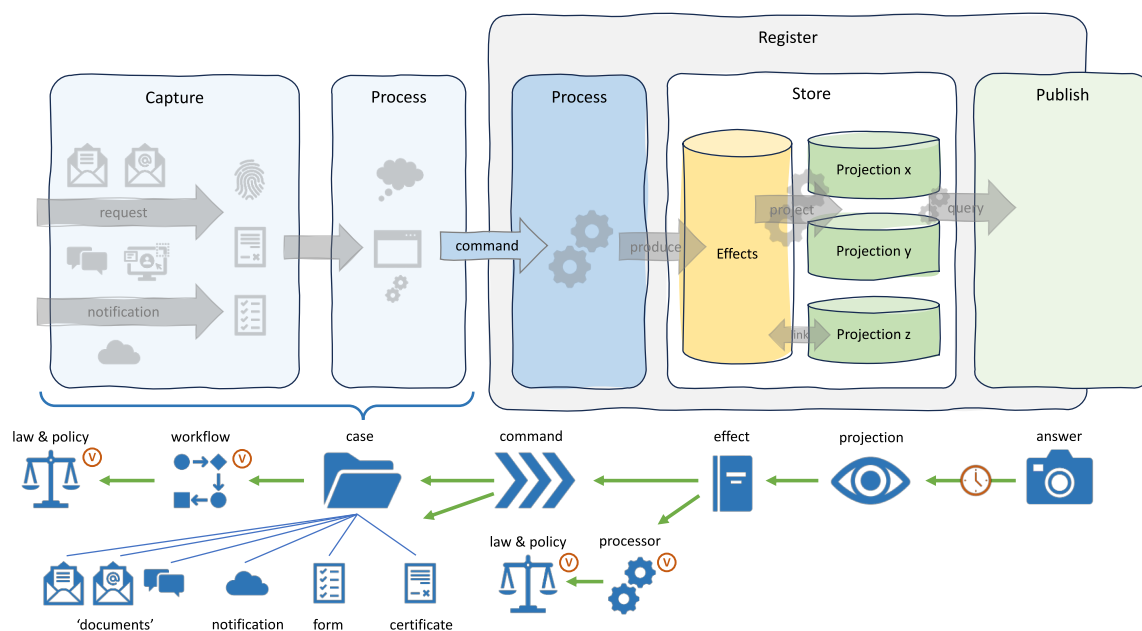


Figure 6.1: The lineage of an answer: from the answer served, back through the projection, the effect, the command and the case, to the captured artefacts - with the processor, the workflows and the law behind them, each versioned.

Figure 6.1 is the architecture of the previous chapter with a second reading drawn underneath. The top half is the now familiar picture, capture to publish, left to right. The bottom half reads against the arrows: it starts from a served answer and traces everything that went into it. The traced path is the answer's *lineage*. Walking it backward answers the provenance question.

The walk starts at the right, with an answer a consumer received. The answer was given against a moment in time. Whoever made a decision on it must be able to ask again - "Where did p5 live on T50, according to what the store held on T135?" - and receive the answer the decision was built on. Chapter 9 works this out as the repeatable question.

All the information in the projection links back to the effect it was projected from. The finer the granularity of the information in the projection, the more accurate the provenance can be.

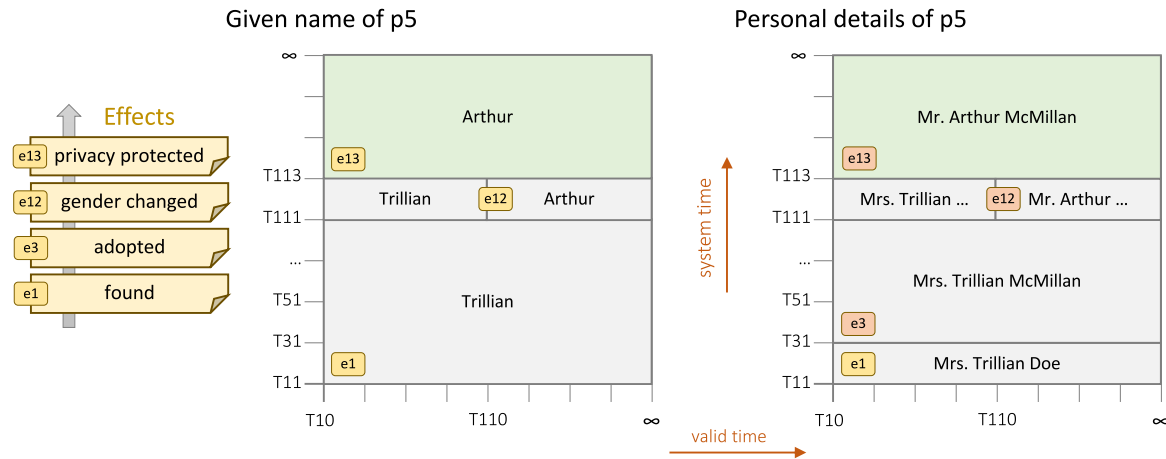


Figure 6.2: Limited provenance: on a grouped table, an effect explains only some of the fields in the row it wrote.

If information is present in the sixth normal form, such as a table with only the `given name`, every information atom can be linked to the effect that brought it there. If attributes are grouped - a `personal details` table that holds `given name`, `family name` and `salutation` - then the row links to the last effect that changed the state. That effect might have been the adoption, which changed the `family name` and not the `given name` and `salutation`. This is not evident from the table itself: it has to be derived from the effect, or by comparing rows to see what changed.

An effect carries two links. One points at the command: the request, with its intent. The other points at the processor, with its exact version, that turned the command into the effect. Code can be flawed. A doubt about an answer may turn out to be a doubt about the algorithm that produced it. So the version of the algorithm matters.

Processors follow law and policy. Normally the lineage between the processor - and its version - and the law and policy behind it can be made on the type level. But sometimes the command has to carry explicit choices about the rules that were applied. In our worked example this could be the case with the nationality. When the parents are not Dutch, or hold multiple nationalities, deriving the nationality of a newborn can be challenging, and the applied clause - perhaps even the reasoning behind choosing it - must be recorded. Such exceptions seep into the command structure; chapter 9 returns to this.

A command rarely arrives out of nowhere. It starts within the capture 'environment'. The case and possibly the individual artefacts - letters, emails, conversations, filled forms, official documents - can be linked to the command.

Cases follow workflows, and workflows are versioned like processors. Behind them, again, stands versioned law and policy. Versioned law is not hypothetical: wetten.overheid.nl, for example, serves law as it stood on a chosen date.

5.2 In practice

How far along this path a register is differs per register, and we have not surveyed all of the core registers; so what follows are a few examples, to the best of our knowledge.

- The BRP carries two kinds of source reference: the *akte* - the civil-status certificate - and a document. Unless a correction is made on the certificate, corrections are often not grounded in such a document.
- The BAG, designed later, made the document a rule: no change to an object without a source document.

- The Kadaster holds two document kinds: the source document that arrived from outside - the notary's deed - and the transaction document that accounts for the processing. Normally one document plays both roles; on a correction they part.

The wish for more elaborate provenance is visible:

- in the case for legal protection around automated chain decisions (Eck, 2018);
- in the documented failures: organisations that cannot give the reasoning behind decisions that follow from changes in a register outside their control (A. C. Widlak & Peeters, 2025, p. 7), origins that were not kept and cannot be traced back (A. Widlak, 2022, p. 105), one register holding 270,000 people without being able to say, per person, why (A. Widlak, 2022, p. 13);
- in the *Handreiking Datalineage* (2025);
- and in the Dutch Algoritmeregister.

5.3 Corrections

In a bitemporal table, corrections make rows with a single link tricky:

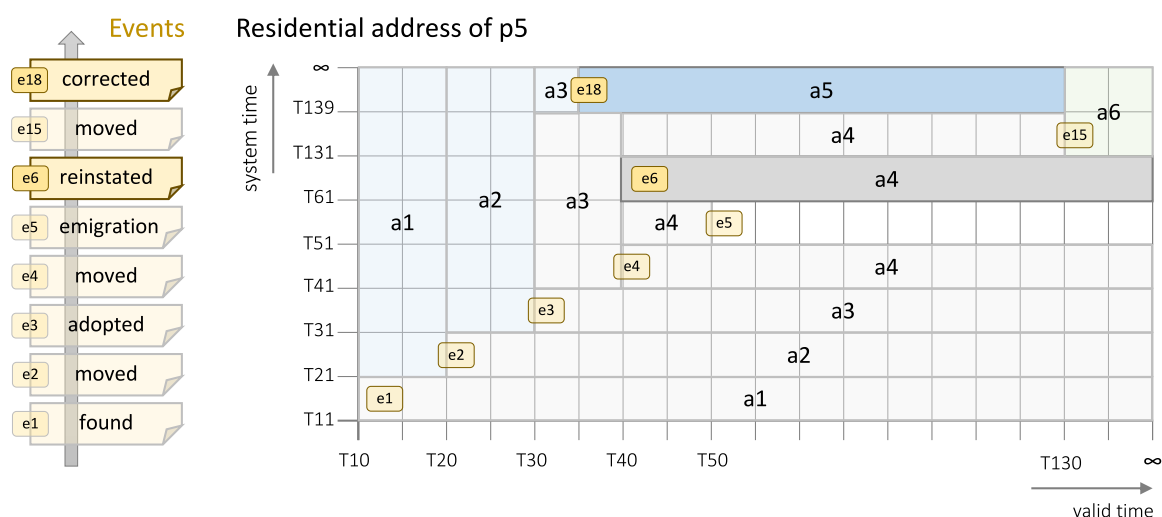


Figure 6.3: The provenance of a correction: after the reinstatement, both highlighted rows point at the correction.

Figure 6.3 shows the residence history around the wrongly ended a4. The emigration (e5) ended it, the correction (e6) reinstated the address. The rows that stand after the address was reinstated were written by e6 - the correction - and their link says so. Ask “why did p5 live at a4?” and the provenance answers “because of a correction”. That is true for the processing, but it does not ground the value.

One way to work around this is to let the row always carry the effect that grounded the state, next to an optional effect responsible for a later operation on it, such as a correction. The row then answers both questions: why this value, and what last touched it. This is not unlike the mechanism of the BAG and the BRK, only applied to effects, not documents.

5.4 Transition

Again we can envision a transition.

1. If there is no lineage at all, add pointers to documents - or commands that point to documents.

2. Start demanding those documents.
3. Separate the grounding of the state from the optional grounding of the last operation.
4. Finally, transition to effects.

5.5 Notes

Three notes for the careful reader.

Exposure. Whatever provenance is internally, its *exposure* has standards - PROV (the W3C model), Dublin Core (general metadata), MDTO (the Dutch archival standard), IMX (PROV-based exchange - per-datum sources and rules).

Projections and authorisation. Not everyone may see provenance. The deeper the lineage goes - certainly where it reaches the cases - the more of it is for authorised employees only. With dedicated projections and appropriate authorisation we can solve the common cases. In the design this can mean that, while certain documents are linked to a case, they are also directly linked in the command, alongside the reference to the case. Given that they are stored in a document management system, we can access them without access to the case.

Verifiable provenance. Provenance can be made *verifiable* - signed attestations, non-repudiation. That is an integrity layer over the links.

6 Doubt

A register is not sure of everything it holds, and a reliable register is explicit about that. This chapter covers what doubt is, how it arrives, how it runs its course, and where it lives in the data. The treatment is light by design: like provenance, doubt is a field of its own; we cover how it impacts the store.

6.1 The flow

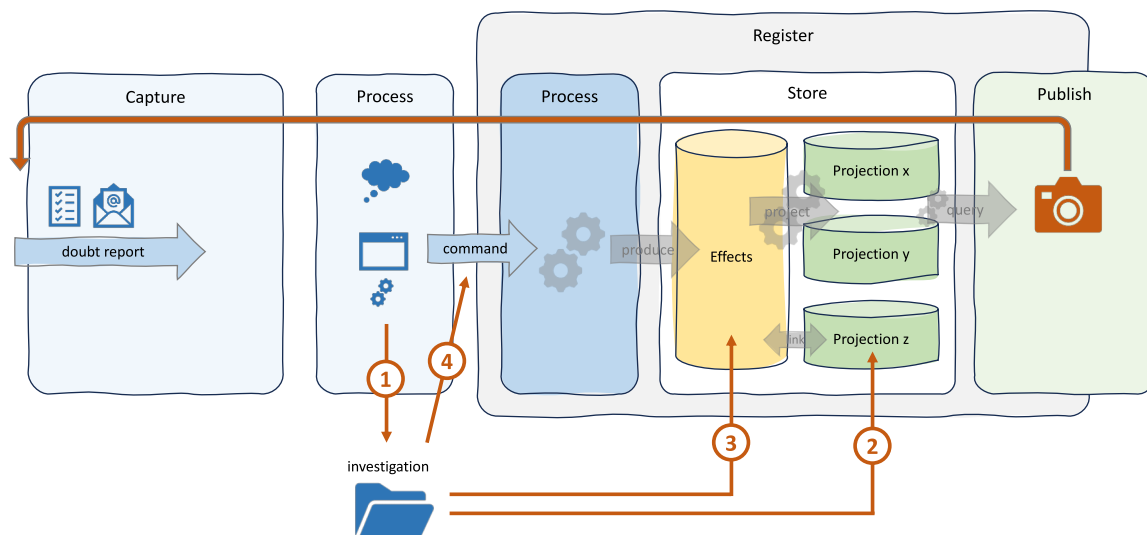


Figure 7.1: The doubt flow

Like the provenance figure, Figure 7.1 reads from the right. A consumer discovers that something in the data might be wrong. The doubt report travels back and arrives at the capture side, on the left. Processing then makes a first determination: is the doubt grounded? A small preliminary check decides whether the report becomes an investigation (1). If it does, a case is opened on the processing side - and that case lives outside the register.

As soon as the investigation exists, consumers need to be informed. If there is a quick fix, the register can skip that step: fix what is wrong and publish the correction instead of the investigation. If more time is needed, the register must first show that there is an investigation. And that is where it gets hard. The consumer saw data in a projection, and it can be hard to find out which effect - or combination of effects - is responsible for what the consumer saw.

In that case we need to mark the data in the projection as under investigation (2). That is a puzzle in and of itself: we bypass the effects to change a projection.

Once the cause is found - the investigation now tied to the responsible effects (3) - a new command adds correction effects (4). In practice those come in at least three shapes.

- The first: a single effect is responsible for the error. We add a correction effect that carries the effect as it should have been. We then replay the effects starting with the replaced one, determine the state as it should have been, compare it to the store, and apply bitemporal operations until the standing records match the desired state.
- The second: it is too hard to isolate a single effect - or the effect can be isolated, but everything after it is impacted: a ripple. Then the correction reasserts all the effects from the incorrect one to the present. This

too leads, eventually, to a comparison with the standing records, and to bitemporal operations until they match the desired state.

- The third: a correction that we cannot yet express in an effect. In most domains it is impossible to know all corrections beforehand - even a register built entirely on law and policy can be overruled by a judge. That is a problem for a functional command API: suddenly there is a correction it has no command for, and possibly no effect either. Chapter 9 returns to this.

A puzzle that remains throughout this: what to do with notifications that should have been sent? The current idea is that non-trivial corrections should be processed manually by consumers.

6.2 Data structure

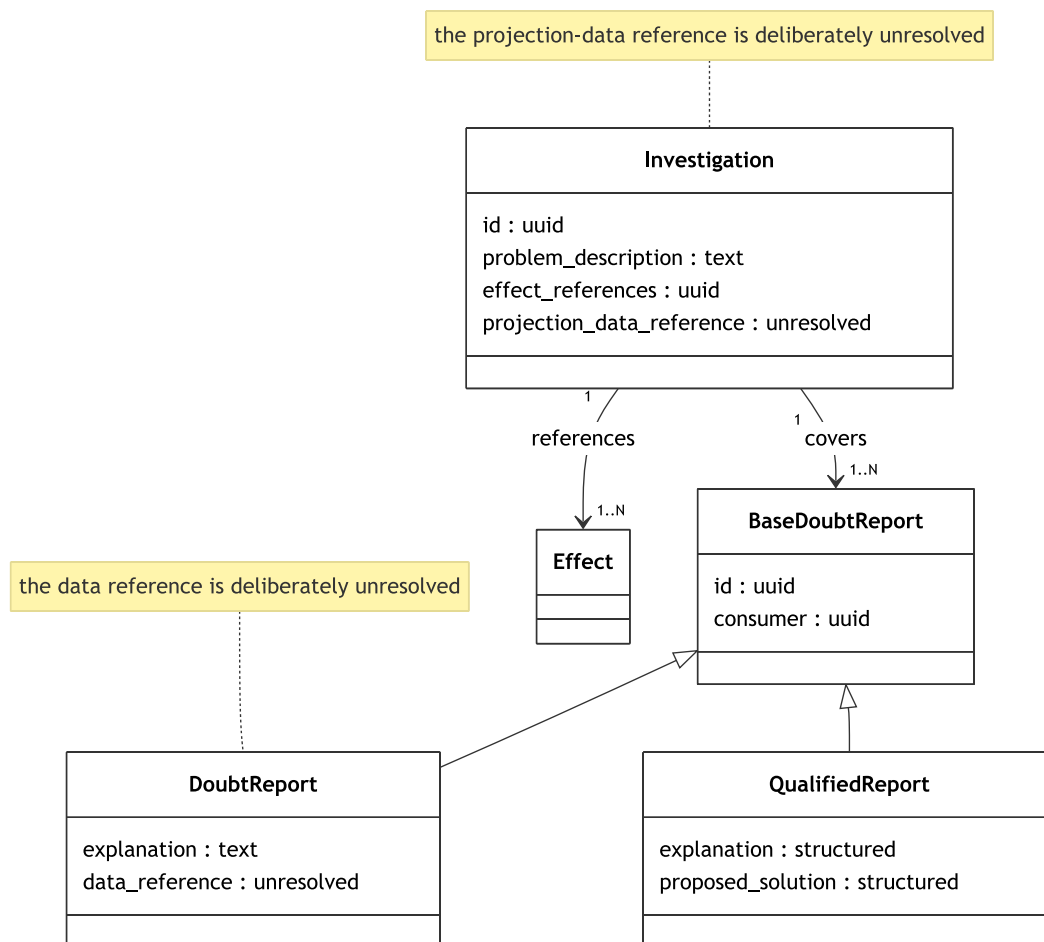


Figure 7.2: The report and the investigation: one supertype, two report kinds, and the investigation that covers them - two references deliberately unresolved.

Figure 7.2 is a rough sketch of the data. A base report carries an id and the reporting consumer. The *doubt report* adds a free-text explanation and a reference to the doubted data. The *qualified report* adds structure: a typed explanation, and even a proposed solution. The investigation covers one or more reports, describes the problem, and references the responsible effects and the doubted data in the projections. Two references are deliberately left unresolved, because resolving them is a puzzle.

There are several issues with incorporating doubt in a register.

The first is how to reference the data you doubt. Current mechanisms often use a flat key-value list. With those you run into two problems: not all data can be referenced that way, and - the more fundamental one - do we really understand the semantics of pointing? In practice, in non-trivial cases different civil servants would reference different instances, at different levels, or different field values, to express the same doubt.

From a human perspective you would rather have a textual description of what is wrong. The reference can then be limited to an entity and its id: the pointer locates, the prose explains. Practical - but it concerns the lawyers who write the law. They do not like free-text fields in a register, and the concern is understandable: a register preferably doesn't open up to free text that might inadvertently contain any form of tainted language.

The other subtype in the diagram is about common cases that can be caught with structure. There have been experiments with qualified feedback reporting: a consumer that has better address information does not just report the doubt, it hands the address over with the report. In practice this is a specialised command for that situation. Such a command carries the exact semantics of the doubt, and can even carry the suggested data.

6.3 The consumer's deviation

One more puzzle relates to this. In the Netherlands consumers of an authoritative register are obliged to use its values - with one exception: once they have filed a doubt report, they may use the value they believe is correct. That poses a new problem. We want consumers to get the data from the source, not copy it; but a consumer that holds its own version needs a data structure for it, and that is an invitation to keep copying values from the register into the consumer's database. What we would have liked: the reported deviation included in the projection for that consumer - or even stored as a separate effect, worked through all the projections that consumer sees. Currently that is too complicated, so consumers deviate by storing their own version and duplicating part of the register's data model.

7 Bitemporal operations (STUB)

This chapter is not yet written: what follows are the planned subjects, in outline.

Operation	What it does
Create	the first registration of a thing or a value
Change	the world moved: the old value ends, the new one begins
Correct	the register was wrong: the belief is replaced
End	a value ceases to be in effect, nothing follows
Reinstate	a wrong ending is undone - a correction, always allowed
Revive	a real return after a real ending - a modelling permission
Withdraw	a row is withdrawn from the register (soft delete)
Erase	a row is erased (hard delete); the exceptional case - if possible at all

8 APIs (STUB)

This chapter is eventually going to be filled by the historie werkgroep of the NL API Strategie. Below is a short inventory of potential topics.

8.1 Commands

8.1.1 Functional APIs

- What shape do functional APIs take, REST, GraphQL, something else? And what shape within REST or GraphQL?
- How do we include the capture side? How do we link to cases and artefacts in a DMS?
- The information from the capture side might not apply to all the information in a command. Some commands are complex and have several subsections that must be processed as one business transaction yet potentially carry different provenance and might come into effect at different times.
- We may have to include references to law and legislation - as part of the provenance - for those situations where the default does not suffice (the nationality example from chapter 6)

8.1.2 Conversational APIs - The impact of doubt

- We want to defer as much processing as we can to the register. Certainly where there are multiple clients. But what if we have rules or an algorithm that encounters doubt? It is practically impossible to design algorithms that foresee every situation. So we need a generic mechanism to deal with doubt. In practice it's feasible to detect if a rule used data that was under investigation. When that happens the severity of the rule should be lowered. Even a hard rule might become one that can be overruled. The decision to overrule lies with the civil servant, not with the register. But that means that the register and the civil servant need to be able to communicate about this. It's not just a command, but a back and forth.

8.1.3 Business transactions - The impact of corrections

- We argued in earlier chapters that we cannot know all corrections beforehand. So we need a mechanism to deal with that. Several solutions have been proposed (and some built):
- If you have a DevOps setting and the law allows you enough processing time, you can build new commands and effects just in time.
- If your register uses functional decomposition on the inside (an adoption is a change of parents, potentially a change of family name, a change of address etc.), you might expose those smaller (decomposed) commands to the outside, perhaps only as part of a wrapping correction command.
- You could do the same with effects: you compose your command of the effects that you would have liked to have.
- You could resort to offering direct access to bitemporal operations, only within a correction wrapper.

- All corrections must take place as a business transaction. They should be handled as a single operation in processing and in publication. Consequences can be severe if this is omitted (notifications will fire on temporarily inconsistent states with potentially catastrophic consequences).

8.1.4 Dry-run APIs - The impact of corrections

- Correcting the past can be daunting. If you - as an IT professional - found the store chapter (chapter 4) hard - you are in good company around the world. Imagine how this is for a domain expert. So we might need dry runs to slowly build the correction (as described in the previous paragraph) and see how it plays out in a projection. This is another type of conversational API.

8.1.5 Bitemporal operations

- We might want APIs that offer bitemporal operations (chapter 8) on the inside and for some registers perhaps also on the outside (or for the corrections).

8.2 Publication

8.2.1 Repeatable question

- Systems normally run multiple write requests in parallel. So if a consumer asks a question, how do we know that what we have just read won't change in a few milliseconds? The practical answer: find the most recent moment where no writers were active. Sounds doable, is rather hard in practice because we don't know the commit times of transactions. (see chapter 12, the concurrency subject). One thing is sure: to be repeatable, a question asked at T100 might very well return the answer that was in the store at T80 because no writers that started before T80 are still active.
- That means we need to return this moment to the consumer: "We answered your question at T80."
- But that might expose unwanted information: The last moment of change.
- So perhaps we should return a token.
- If that token is specific to the consumer, that might also solve an authorisation problem that we have. Lawmakers want the impossible: accountability and a real erase. We can't do both so we offer a soft delete. That soft delete should still return what once was for repeatable questions (we think, up for debate). If it should, we can limit such questions to those who actually asked them before, because the token is the proof that they did.

8.2.2 Information products

- If we really embrace projections, we can come closer to the needs of consumers. Where we now expose (rather than publish) databases, we can expose information in denormalised form, etc. Perhaps other types of APIs like GraphQL are more suitable for such APIs.

8.2.3 Notifications

- Notifications are projections too - by design. A lot of consumers are not allowed to be notified about adoptions, but they are allowed to be informed about a changed family name or a changed address.

9 Design part I - Concepts and building blocks (STUB)

This chapter is not yet written: what follows are the planned subjects, in outline.

- Designs build on the interval form (S4). We don't design for S5.
- Identity: Of entities, weak entities, relations.
- Granularity: which fields to group in a historised table. Between BCNF and 6NF.
- Gaps and overlaps
- Pattern fields: the fields that are added to a historised table.

10 Design part II - Table patterns (STUB)

This chapter is not yet written: what follows are the planned subjects, in outline.

- The anchor: the non-temporal identity table everything hangs off.
- The lifespan table: existence and 'in effect' without a second authoritative interval.
- Single-value attributes
- Relations and multi-value attributes
- Event records: happenings with a happened_on moment instead of a valid pair.
- Reference tables: the maintained code lists (stamgegevens).

11 Deferred topics (STUB)

This chapter is not yet written: what follows are the planned subjects, in outline.

- The future: future-dated data - a `valid_from` ahead of today.
- Concurrency and time: simultaneous writes, and what they do to the repeatable answer.
- Re-synchronisation: bulk repair against a source.
- Schema versioning: the store's own shape changing over time.

12 Terminology

Terms used in this document, with their working descriptions. This chapter is not finished: in the final version every term is weighed against its alternatives.

A term in *italic* has its own entry; literal field names are shown in code.

Term	Description
Availability	A word used only in the publication context: the <i>projection's</i> own system time, read by a consumer, tells when data was available. Not a third axis.
Bitemporal	A table that carries both time axes: <i>system time</i> and <i>valid time</i> .
Chronon	The smallest unit of time a store distinguishes; a timeline is a sequence of chronons, and nothing falls between two of them.
Command	An instruction to the register, carrying the client's intent.
Datum	A single unit of data the <i>register</i> holds.
Effect	The recorded result of processing a <i>command</i> : a statement of what the register did, appended to the journal and never altered.
Erase	Physically destroying a row; the exceptional case.
Event	In this document: a real-world happening.
Field	A named slot that holds a <i>datum</i> .
Historised table	A table that may spread the information about one thing over several rows - one row per period - where a normal table holds a single row per thing. Prefixed <code>his_</code> : in such a table, rows are not things.
Invalid	A word this document does not use: a period that ended is not wrong, and a wrong row is not ended.
Lineage	The traced path from a served answer back to everything that went into it: projection, effect, command, case - and the versioned processors, workflows and law behind them. Provenance is the question; lineage is the path.
Marker row	A row in a point-based table that asserts no new value, but marks what happened to an old one: ended, wrong, or hidden.
Projection	A view on a register from the perspective of a certain (class of) use case(s). Derived from the effects.
Register	An application component - or a set of cooperating application components - for reliably recording and presenting an ordered collection of government information.
Standing record	The row that still stands in a <i>historised table</i> for a thing: the row whose <code>system_until</code> is open.
Supersede	Replacing a row with a newer one on the system axis, keeping the old.
System time	A time axis: it records when a <i>datum</i> was stored in the <i>register</i> , and when it ceased to stand. The literature says transaction time; SQL:2011 and this document say system time.
<code>system_from</code> / <code>system_until</code>	The two fields that carry a row's system-time period: <code>system_from</code> when the row was recorded, <code>system_until</code> when it ceased to stand - superseded, withdrawn or erased. Half-open; an open <code>system_until</code> marks the <i>standing record</i> .
Unitemporal	A table that carries exactly one time axis - either <i>system time</i> or <i>valid time</i> .
Valid time	A time axis: it records when a value is in effect in the domain, from when it begins to when it ends.
<code>valid_from</code> / <code>valid_until</code>	The two fields that carry a row's valid-time period: <code>valid_from</code> when the value comes into effect in the domain, <code>valid_until</code> when it ceases to be in effect. Half-open; not assigned by the store - they are determined by the domain.
Withdraw	Soft delete.

Appendix A: Bibliography

- Basisregistratie Ondergrond. (2017). *Programma Start Architectuur Basisregistratie Ondergrond* [Programma Start Architectuur]. https://basisregistratieondergrond.nl/publish/pages/162257/20-11-2017_programma_start_architectuur_psa.pdf
- Bohlen, M. H., Dignos, A., Gamper, J., & Jensen, C. S. (2017). *Temporal Data Management – An Overview*.
- Böhlen, M. H., Dignös, A., Gamper, J., & Jensen, C. S. (2018). Database Technology for Processing Temporal Data (Invited Paper). *LIPICs, Volume 120, TIME 2018, 120*, 2:1–2:7. <https://doi.org/10.4230/LIPICS.TIME.2018.2>
- Date, C. J., Darwen, H., & Lorentzos, N. A. (2003). *Temporal data and the relational model: A detailed investigation into the application of interval and relation theory to the problem of temporal database management* (1st ed). Morgan Kaufmann Publishers.
- Date, C. J., Darwen, H., & Lorentzos, N. A. (2014). *Time and relational theory: Temporal databases in the relational model and SQL*. Morgan Kaufmann.
- Eck, B. M. A. van. (2018). *Geautomatiseerde ketenbesluiten & rechtsbescherming: een onderzoek naar de praktijk van geautomatiseerde ketenbesluiten over een financieel belang in relatie tot rechtsbescherming*. Tilburg University.
- Fowler, M. (2011, July 14). CQRS. <https://martinfowler.com/bliki/CQRS.html>
- Geonovum. (2020). *IMGeo Catalogus BGT*. <https://docs.geostandaarden.nl/imgeo/catalogus/bgt/>
- Jensen, C. S., Dyreson, C. E., Böhlen, M., Clifford, J., Elmasri, R., Gadia, S. K., Grandi, F., Hayes, P., Jajodia, S., KÄfer, W., Kline, N., Lorentzos, N., Mitsopoulos, Y., Montanari, A., Nonen, D., Peressi, E., Pernici, B., Roddick, J. F., Sarda, N. L., ... Wiederhold, G. (1998). The consensus glossary of temporal database concepts — February 1998 version. In O. Etzion, S. Jajodia, & S. Sripada (Eds.), *Temporal Databases: Research and Practice* (Vol. 1399, pp. 367–405). Springer Berlin Heidelberg. <https://doi.org/10.1007/BFb0053710>
- Jensen, C. S., & Snodgrass, R. T. (Jan.-Feb./1999). Temporal data management. *IEEE Transactions on Knowledge and Data Engineering*, 11(1), 36–44. <https://doi.org/10.1109/69.755613>
- Jensen, C. S., Soo, M. D., & Snodgrass, R. T. (n.d.). *Unifying Temporal Data Models*.
- Kadaster. (2018). *Specificatie BAG Historiemodel*. <https://www.kadaster.nl/-/specificatie-bag-historiemodel>
- Kadaster. (2025, October 16). *Kadaster historiemodel 4.3.0*.
- Kadaster. (2026, May 18). *Informatiemodel BRT (IMBRT)*. <https://kadaster.github.io/imbrt/>
- Kamer van Koophandel. (2022). *Gegevenscatalogus Handelsregister (KvK Dataservice)*. <https://www.kvk.nl/producten-bestellen/kvk-dataservice-aansluiten-gegevenscatalogus/>
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd edition). Wiley.
- Ministerie van Binnenlandse Zaken en Koninkrijksrelaties. (2017). *Documentatie bidirectionele conversie* [Programmadocumentatie Operatie BRP]. <https://github.com/MinBZK/OperatieBRP/blob/master/01%20Documentatie/05%20Migratie/01%20Functioneel/Documentatie%20bidirectionele%20conversie.pdf>
- Müller, M. (n.d.). *Enabling Retroactive Computing Through Event Sourcing*.

- NORA Expertgroep Gegevensmanagement. (2025). *Handreiking Datalineage: inzicht in de herkomst van gegevens* [Handreiking].
- Overeem, M., Spoor, M., & Jansen, S. (n.d.). *The dark side of event sourcing: Managing data conversion*.
- Overeem, M., Spoor, M., Jansen, S., & Brinkkemper, S. (2021, April 2). *An Empirical Characterization of Event Sourced Systems and Their Schema Evolution – Lessons from Industry*.
<https://doi.org/10.1016/j.jss.2021.110970>
- Peeters, R., & Widlak, A. (2018). The digital cage: Administrative exclusion through information architecture – The case of the Dutch civil registry’s master data management system. *Government Information Quarterly*, 35(2), 175–183. <https://doi.org/10.1016/j.giq.2018.02.003>
- Petković, D. (2016). Support of Temporal Data in Database Systems. *International Journal of Computer Applications*, 152.
- Rijksdienst voor Identiteitsgegevens. (2026). *Logisch Ontwerp BRP*. <https://www.rvig.nl/lo-brp>
- Rönnbäck, L., Regardt, O., Bergholtz, M., Johannesson, P., & Wohed, P. (2010). Anchor modeling — Agile information modeling in evolving data environments. *Data & Knowledge Engineering*, 69(12), 1229–1253. <https://doi.org/10.1016/j.datak.2010.10.002>
- Snodgrass, R. T. (2000). *Developing time-oriented database applications in SQL*. Morgan Kaufmann.
- Temporal database. (2026). In *Wikipedia*. https://en.wikipedia.org/wiki/Temporal_database
- VNG, U. (2026). *Website Uit Betrouwbare Bron*. www.uitbetrouwbarebron.nl
- W3C. (2013). *PROV-DM: The PROV data model* (L. Moreau & P. Missier, Eds.) [W3C Recommendation]. World Wide Web Consortium. <https://www.w3.org/TR/prov-dm/>
- Waarderingskamer. (2024). *Catalogus Basisregistratie WOZ* (Version 1.8) [Gegevenscatalogus].
<https://www.waarderingskamer.nl/documenten/03.-Voor-gemeenten/04.-Gegevensbeheer/Catalogus-Basisregistratie-WOZ-versie-1.8.pdf>
- Widlak, A. (2022). *Volwassen Digitale Overheid* (1st ed). Boom Uitgevers Den Haag.
- Widlak, A. C., & Peeters, R. (2025). A theory of the infrastructure-level bureaucracy: Understanding the consequences of data-exchange for procedural justice, organizational decision-making, and data itself. *Government Information Quarterly*, 42(2), 102021. <https://doi.org/10.1016/j.giq.2025.102021>
- Young, G. (2010, November). *CQRS documents*.
https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf